

Audience: LEWG, SG14, WG21
Document number: P0447R26
Date: 2023-12-17
Project: Introduction of std::hive to the standard library
Reply-to: Matthew Bentley <mattreecebentley@gmail.com>

INTRODUCTION OF STD::HIVE TO THE STANDARD LIBRARY

Table of Contents

- I. [Introduction](#)
- II. [Definitions](#)
- III. [Motivation and Scope](#)
- IV. [Impact On the Standard](#)
- V. [Design Decisions](#)
- VI. [Technical Specification](#)
- VII. [Acknowledgments](#)
- VIII. Appendices:
 - A. [Basic usage examples](#)
 - B. [Original reference implementation benchmarks, differences from current reference, and links](#)
 - C. [Frequently Asked Questions](#)
 - D. [Typical game engine requirements](#)
 - E. [User experience reports](#)
 - F. [Brief guide for selecting an appropriate container based on usage and performance](#)
 - G. [Hive constraints summary](#)
 - H. [Links to prior art](#)
 - I. [Information on non-reference-implementation hive designs](#)

Revision history

- R26: Tech spec changes: kona final meeting changes applied: `is_active()` removed. Precondition added to `get_iterator`, Effects 'return value of `end()` when pointer is not in `*this`' clause is removed. Heading corrections. Formatting corrections.
Non-tech-spec changes: Update google groups links for new archived google groups format. Bring prose sections of the paper up to date with the finalized technical specification and reduce those sections' repetition. Move alternative implementation guidelines from Design Decisions to the alt implementation appendix. Frequently Asked Questions and "responses to specific questions from the committee" combined into single appendix. Benchmarks and notes re: original reference implementation vs current implementation combined into a single appendix. "Questions for the committee" section removed, replaced with (brief) Definitions section. Links moved from `plflib.org` to external, potentially more-permanent sites where possible. Some FAQ entries consolidated/removed. References to `distance()` being able to deal with negative values removed (user must use iterator comparison operators and swap where necessary). Some corrections to FAQ. Info about lack of `resize()` moved from Design Decisions to FAQ. Time complexity appendix removed and info moved into Design Decisions section for individual functions, to avoid repetition. Changed 'over-alignment' to 'artificial widening the storage' as it is more accurate. Changed "bitfield + jump-counting" approach in alt implementation appendix to cover smaller types. Added 3 better approaches to 'supporting very small types' under alt implementation appendix. Corrected some links. Add section about comparison to slot maps to FAQ.
- R25: Publication of mid-Kona intra-conference draft updates (based on committee feedback) for book-keeping: Update to reshape's exception guarantees. Add 'invalidates past-the-end iterators' for `splice`. Add `constexpr` to definition of `block_capacity_limits()`. Reduction/clarification of when element reordering may occur for reshape, `shrink_to_fit`. Slight rewording of `sort()` to be more in-keeping with `list::sort()`. Minor editorial corrections ("shall be" to "is", "into hive" to "into `*this`") to match current standard ie. "T shall be `Cpp17MoveInsertable` into hive" becomes "T is `Cpp17MoveInsertable` into `*this`". Added "This operation may change capacity()" to remarks in reshape. `sort`: Throws section removed as this is covered by blanket wording and by Remarks and the post-Remarks note. Specifically the following line is removed: "Throws: `bad_alloc` if it fails to allocate any memory necessary for this operation. `comp` may also throw." Note: `forward_list::sort` and `list::sort` do not include such a line.
- R24: Corrected HTML errors (some highlighting incorrect in tech specification, other places). Overhauled info for supporting small types without overalignment, in the alt implementation details appendix. Minor corrections to tech spec. Slight rewording of `shrink_to_fit` tech spec to be closer in wording to vector's.
- R23: Correction/update to `constexpr` usage section in appendices. Correction of `bit_cast` to `reinterpret_cast` in Design Decisions. Added section within Design Decisions->Erased-element location recording mechanism, detailing the various approaches possible for keeping track of which blocks contain erasures. Correction to Design Decisions->Collection of element blocks + metadata. Addition of explanatory infographic to intro, courtesy of Victor Reverdys suggestion. `<=>` exclusion note removed from tech overview. Growth factor "greater than 1" removed from overview and "which need not be integral" added to bring in line with other mentions of growth factor in Standard. Note1 'poem' removal in hive.overview taken out, referred to LWG directly for review. LWG feedback on time complexity wording: make maximum required by current implementation, if at later point it needs to be adjusted it can, future ABI breakage is more a concern than time complexity. Time complexity for erasure-handling removed as it is possible to make it $O(1)$ for all types without overaligning small types (see end of alt implementation appendix for details). Tim Song and Jens Maurer signed off on erasure-handling time complexity wording ($x + y$ where x = in elements, y = in element blocks, this approach is mirrored in parts of the standard). Further updates based on Ben Craig's feedback & private review group feedback. Other corrections to time complexity. Updates to alt implementation details appendix. Removed `clear()` description from tech spec as is covered by the containers blanket wording and the overview blanket wording. Erase descriptions also reduced as portions were covered by overview blanket wording.
- R22: Addition of Hive constraints summary in appendices. Addition of prior art info to appendices. Additional information around alternative (vector-of-pointer based) implementations added in Appendices and said information in Design Decisions modified. Some other appendix items updated.
- R21: Included note in Design Decisions section regarding conditions under which block capacity limits are copied between hives, and formalized this in the Technical Specification. Corrections to time complexity appendix. Correction to title in Design Decisions section.
- R20: Removal of `==` `!=` and `<=>` container operators. Reasoning for this added to the FAQ appendix. Addition of reference implementation licensing compatibility with other licensing to FAQ appendix. Minor corrections. Removal of complexity specification for `sort()` to allow for different algorithms to be used. Iterator invalidation information

moved into tech spec function descriptions. Tech spec overhaul via Ben Craig's feedback. C++20 ranges overloads added. Tech spec numbering removed, replaced with tags. Addition of 'block_capacity_hard_limits' function. More FAQ entries. Remove priority template parameter and reasoning added to FAQ. Tech spec overhaul via Jonathan Wakely's feedback. Removal of memory() (see FAQ), trim() renamed to trim_capacity(). Conditions for functions with block limits as an argument changed from throwing when not satisfying requirements, to undefined behavior (now that requirements can always be satisfied by user via calling block_capacity_hard_limits()). Addition of unique() functions since optimal implementation is non-intuitive. Clarification of erase() iterator invalidation rules. iterator get_iterator(pointer p) changed to iterator get_iterator(const_pointer p). Corrections to synopsis. Removal of advance/distance/next/prev overloads from tech spec (this allows them to be specialisations within those functions for implementors). Removed copy constructors and operator= from [hive.cons] as these are covered in [sequence.req] (this is reflected in other sequence container [cons]). hive_limits constructor changed to constexpr to allow for constexpr calling from block_capacity_hard_limits(). is_active(const_iterator) added. reverse_iterator and const_reverse_iterator now equivalent to std::reverse_iterator<iterator/const_iterator> in the tech spec. Corrections/additions to time complexity details in tech spec. Tech spec update based on Tim Song's feedback and committee feedback. trim_capacity(n) overload added. hive_limit constructor defaults changed to separate overloads.

- R19: Correction to intro. Addition of sort() to invalidation rules. Removal of questions for the committee based on Gašper Ažman's feedback. Minor corrections. Moved constexpr explanation to appendices. Addition and removal of questions from appendices.
- R18: Addition of <=> operator. Addition of basic guide for container selection within/without the standard library, in appendix. Addition of edits to [containers.general] and [sequence.reqmts] in technical specification. Update 22.3.14.1. Some rewording.
- R17: Addition of appendix containing reported user experiences. Editing of constexpr exploration.
- R16: Explanation of desired clear() behavior added in Design Decisions section. References to colony reference implementation changed to refer to hive reference implementation (C++20 only). Textual corrections. Range-constructor corrected to allow sentinels.
- R15: Added throw details to splice. Further design decisions information on reshape and splice. Assign() overload for sentinels (differing iterator types) added. Minor text snafu corrections. Colony changed to hive based on D2332R0.
- R14: get_iterator_from_pointer changed to get_iterator - the pointer part is implied by the fact that it's the only argument. Added const_iterator overload for get_iterator - which takes a const_pointer and is a const function. Some wording corrections, additional design decisions information. HTML corrections.
- R13: Revisions based on committee feedback. Skipfield template parameter changed to priority enum in order to not over-specify container implementation. Other wording changes to reduce over-specifying implementation. Some non-member template functions moved to be friend functions. std::limits changed to std::colony_limits. block_limits() changed to block_capacity_limits().
- R12: Fill, range and initializer_list inserts changed to void return, since the insertions are not guaranteed to be sequential in terms of colony order and therefore returning an iterator to the first insertion is not useful. Non-default-value fill constructor changed to non-explicit to match other std:: containers. Correction to reserve() wording. Other minor corrections and clarity improvements.
- R11: Overhaul of technical specification to be more 'wording-like'. Minor alterations & clarifications. Additional alternative approach added to Design Decisions under skipfield information. Overall rewording. Reordering based on feedback. Removal of some easily-replicated 'helper' functions. Change to noexcept guarantees. Assign added. get_block_capacity_limits and set_block_capacity_limits functions renamed to block_limits and reshape. Addition of block-limits default constructors. Reserve() and shrink_to_fit() reintroduced. trim(), erase and erase_if overloads added.
- R10: Additional information about time complexity requirements added to appendix, some minor corrections to time complexity info. The 'bentley pattern' (this was always a temporary name) is renamed to the more astute 'low-complexity jump-counting pattern'. Likewise the 'advanced jump-counting skipfield' is renamed to the 'high-complexity jump-counting pattern' - for reasoning behind this go [here](#). Both refer to time complexity of operations, as opposed to algorithmic complexity. Some other corrections.
- R9: Link to Bentley pattern paper added, and is spellchecked now.

- R8: Correction to SIMD info. Correction to structure (missing appendices title, member functions and technical specification were conjoined, acknowledgments section had mysteriously gone missing since an earlier version, now restored and updated). Update intro. HTML corrections.
- R7: Minor changes to member functions.
- R6: Re-write. Reserve() and shrink_to_fit() removed from specification.
- R5: Additional note for reserve, re-write of introduction.
- R4: Addition of revision history and review feedback appendices. General rewording. Cutting of some dead wood. Addition of some more dead wood. Reversion to HTML, benchmarks moved to external URL, based on feedback. Change of font to Times New Roman based on looking at what other papers were using, though I did briefly consider Comic Sans. Change to insert specifications.
- R3: Jonathan Wakely's extensive technical critique has been actioned on, in both documentation and the reference implementation. "Be clearer about what operations this supports, early in the paper." - done (V. Technical Specifications). "Be clear about the O() time of each operation, early in the paper." - done for main operations, see V. Technical Specifications. Responses to some other feedbacks included in the foreword.
- R2: Rewording.

I. Introduction

hive

blocks with erasures list head

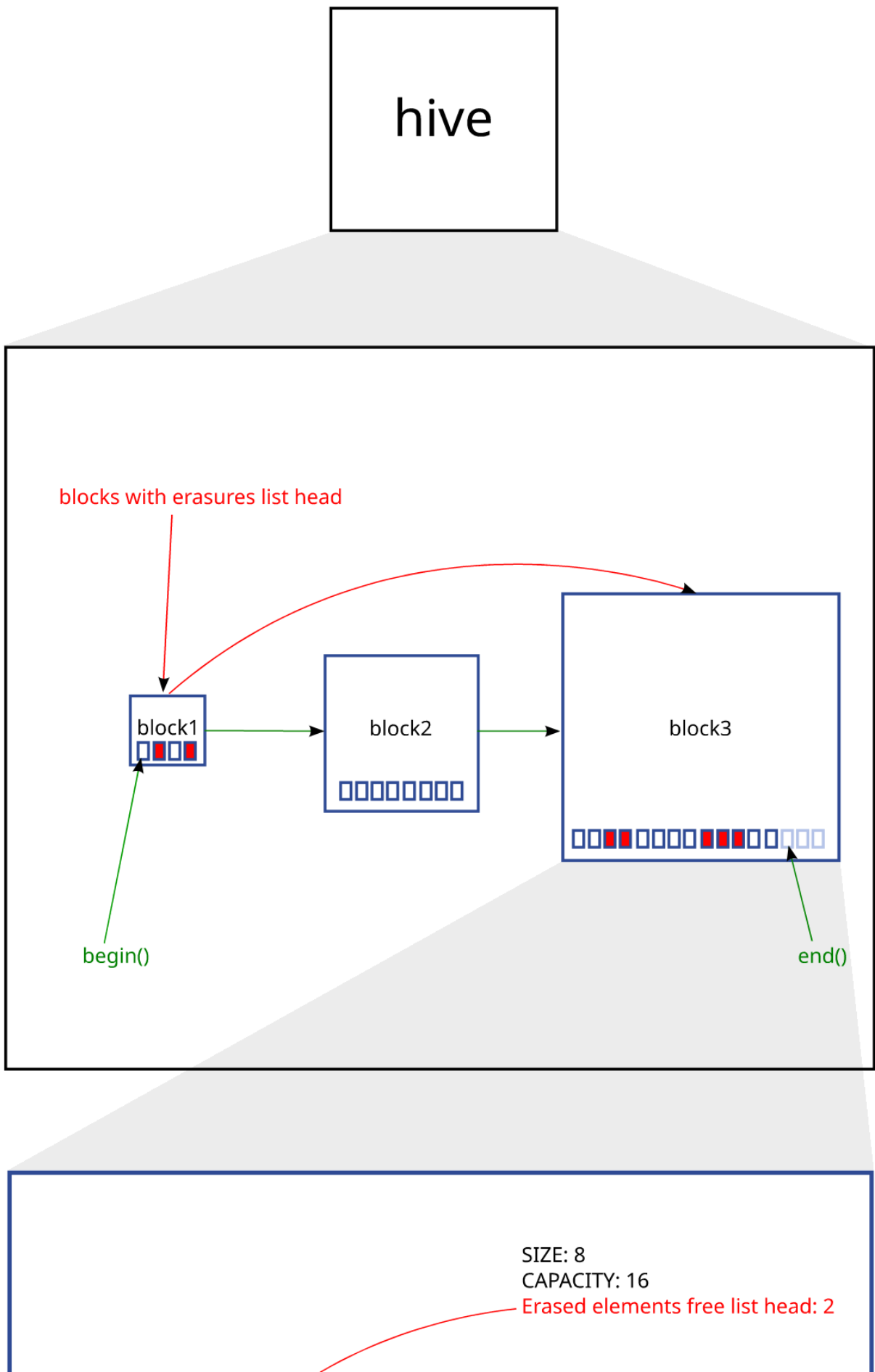
begin()

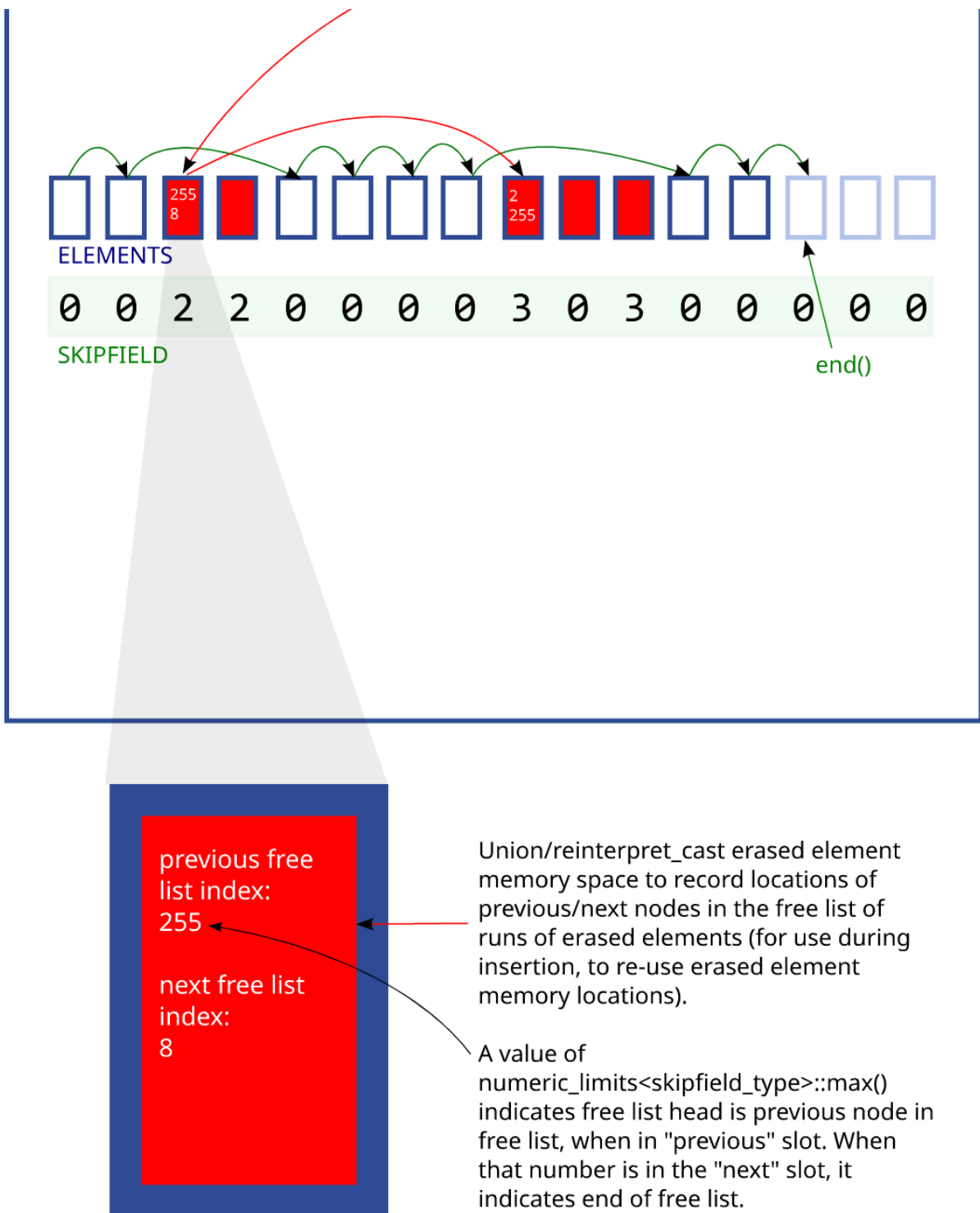
end()

SIZE: 8

CAPACITY: 16

Erased elements free list head: 2





* NOTE: This visualises the approach taken by the reference implementation and is supplied for the purpose of giving a general understanding of structure, but other approaches are possible, see Design Decisions.

The purpose of a container in the standard library cannot be to provide the optimal solution for all scenarios. Inevitably in fields such as high-performance trading or gaming, the optimal solution within critical loops will be a custom-made one that fits that scenario perfectly. However, outside of the most critical of hot paths, there is a wide range of application for more generalized solutions.

Hive is a formalisation, extension and optimization of what is typically known as a 'bucket array' or 'object pool' container in game programming circles. Thanks to all the people who've come forward in support of the paper over the years, I know that similar structures exist in various incarnations across many fields including high-performance computing, high performance trading, 3D simulation, physics simulation, robotics, server/client application and

particle simulation fields (see [this google groups discussion](#), the [hive supporting paper #1](#) and [appendix links to prior art](#)).

The concept of a bucket array is: you have multiple memory blocks of elements, and a boolean token for each element which denotes whether or not that element is 'active' or 'erased' - commonly known as a skipfield. If it is 'erased', it is skipped over during iteration. When all elements in a block are erased, the block is removed, so that iteration does not lose performance by having to skip empty blocks. If an insertion occurs when all the blocks are full, a new memory block is allocated.

The advantages of this structure are as follows: because a skipfield is used, no reallocation of elements is necessary upon erasure. Because the structure uses multiple memory blocks, insertions to a full container also do not trigger reallocations. This means that element memory locations stay stable and iterators stay valid regardless of erasure/insertion. This is highly desirable, for example, [in game programming](#) because there are usually multiple elements in different containers which need to reference each other during gameplay, and elements are being inserted or erased in real time. The only non-associative standard library container which also has this feature is `std::list`, but it is undesirable for performance and memory-usage reasons. This does not stop it being used in [many open-source projects](#) due to this feature and its splice operations.

Problematic aspects of a typical bucket array are that they tend to have a fixed memory block size, tend to not re-use memory locations from erased elements, and utilize a boolean skipfield. The fixed block size (as opposed to block sizes with a growth factor) and lack of erased-element re-use leads to far more allocations/deallocations than is necessary, and creates memory waste when memory blocks have many erased elements but are not entirely empty. Given that allocation is a costly operation in most operating systems, this becomes important in performance-critical environments. The boolean skipfield makes iteration time complexity at worst $O(n)$ in `capacity()`, as there is no way of knowing ahead of time how many erased elements occur between any two non-erased elements. This can create variable latency during iteration. It also requires branching code for each skipfield node, which may cause performance issues on processors with deep pipelines and poor branch-prediction failure performance.

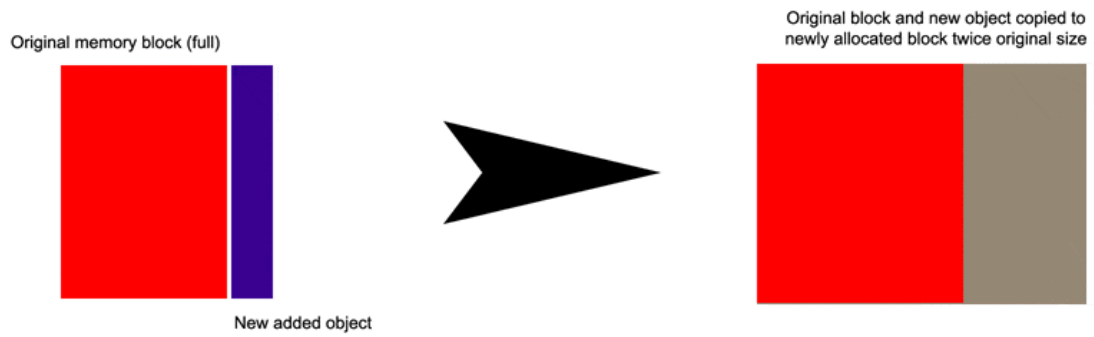
A hive uses a non-boolean method for skipping erased elements, which allows for more-predictable iteration performance than a bucket array and $O(1)$ iteration time complexity; the latter of which means it meets the C++ standard requirements for iterators, which a boolean method doesn't. It has an (optional - on by default) growth factor for memory blocks and reuses erased element locations upon insertion, which leads to fewer allocations/reallocations. Because it reuses erased element memory space, the exact location of insertion is undefined. Insertion is therefore considered unordered, but the container is sortable. Lastly, because there is no way of predicting in advance where erasures ('skips') may occur between non-erased elements, an $O(1)$ time complexity `[]` operator is not possible and thereby the container is bidirectional but not random-access.

There are two patterns for accessing stored elements in a hive: the first is to iterate over the container and process each element (or skip some elements using `advance/prev/next/iterator ++/--` functions). The second is to store an iterator returned by `insert()` (or a pointer derived from the iterator) in some other structure and access the inserted element in that way. To better understand how insertion and erasure work in a hive, see the following diagrams.

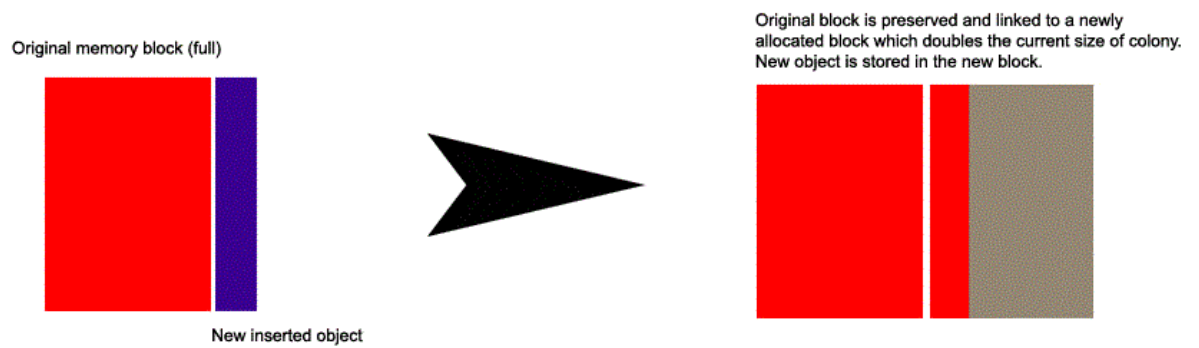
Insertion to back

The following demonstrates how insertion works in a hive compared to a vector when `size == capacity`.

Vector:



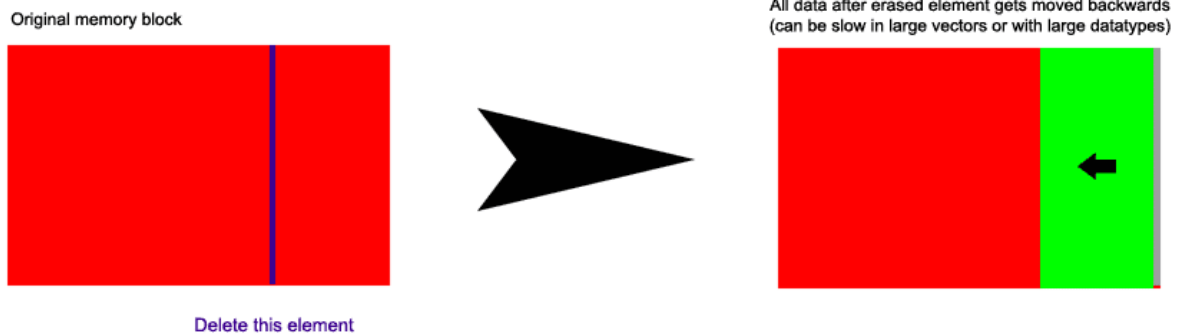
Hive:



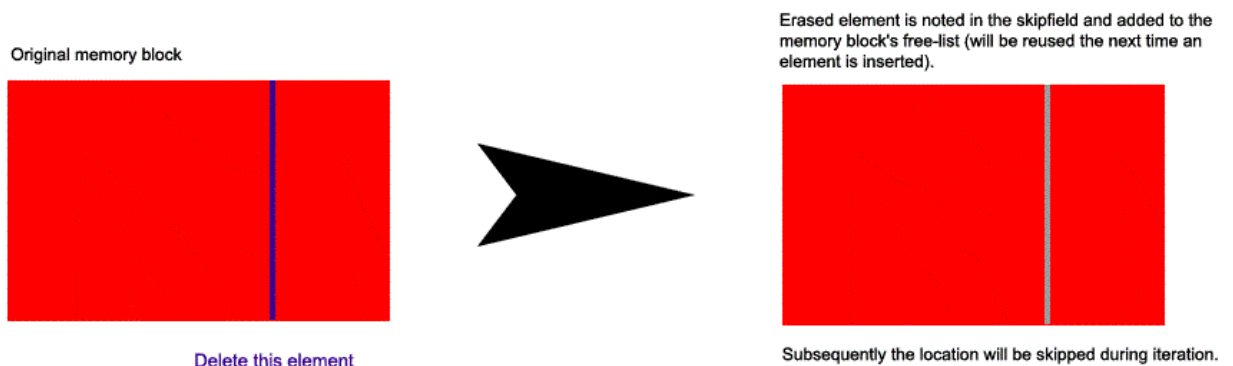
Non-back erasure

The following images demonstrate how non-back erasure works in a hive compared to a vector.

Vector:



Hive:



There is additional introductory information about the container's structure in [this CPPcon talk](#), though much of it's information is out of date (hive no longer uses a stack but a [free list](#) instead, benchmark data is out of date, etcetera), and more detailed implementation information is available in [this CPPnow talk](#). Both talks discuss the precursor for `std::hive`, called [plf::colony](#).

II. Definitions

For the purposes of the non-technical-specification sections of this document, the following terms are defined:

- Link: denotes any form of referencing between elements whether it be via ids/iterators/pointers/indexes/references or anything else.
- Active blocks: memory blocks of type T, which are interacted with when iterating over the sequence in a multiple-memory-block container.
- Reserved blocks: memory blocks of type T, which do not contain elements but which are retained by a multiple-memory-block container for future insertions.
- Element blocks: both active and reserved blocks.
- Skipfield: An array of integers or a bitfield, used to skip over certain objects in an accompanying data structure during iteration or processing. In the context of a container typically used to indicate erased elements.
- Skipblock: a run of elements which are contiguous in memory and designated to be skipped during iteration. In the context of a container, contiguous erased elements.

III. Motivation and Scope

There are situations where data is heavily interlinked, iterated over frequently, and changing often. An example is the typical video game engine. Most games will have a central generic 'entity' or 'actor' class, regardless of their overall schema (an entity class does not imply an [ECS](#)). Entity/actor objects tend to be 'has a'-style objects rather than 'is a'-style objects, which link to, rather than contain, shared resources like sprites, sounds and so on. Those shared resources are usually located in separate containers/arrays so that they can re-used by multiple entities. Entities are in turn referenced by other structures within a game engine, such as quadrees/octrees, level structures, and so on.

Entities may be erased at any time (for example, a wall gets destroyed and no longer is required to be processed by the game's engine, so is erased) and new entities inserted (for example, a new enemy is spawned). While this is all happening the links between entities, resources and superstructures such as levels and quadrees, must stay valid in order for the game to run. The order of the entities and resources themselves within the containers is, in the context of a game, typically unimportant, so an unordered container is okay. More specific requirements for game engines are listed in the [appendices](#).

But the non-fixed-size container with the best iteration performance in the standard library, vector, loses pointer validity to elements within it upon insertion, and pointer/index validity upon erasure. This leads towards sophisticated and often restrictive workarounds when developers attempt to utilize vector or similar containers under the above circumstances.

`std::list` and the like are not suitable due to their poor memory locality, which leads to poor cache performance during iteration. This [does not stop them](#) from being used extensively. This is however an ideal situation for a container such as `hive`, which has a high degree of memory locality. Even though that locality can be punctuated by gaps from erased elements, it still works out better in terms of iteration performance than all other standard library containers other than `deque`/`vector`, regardless of the ratio of erased to non-erased elements (see [benchmarks](#)). It is also in most cases faster for insertion and (non-back) erasure than current standard library containers.

As another example, particle simulation (weather, physics etcetera) often involves large clusters of particles which interact with external objects and each other. The particles each have individual properties (eg. spin, speed, direction etc) and are being created and destroyed continuously. Therefore the order of the particles is unimportant, what is important is the speed of erasure and insertion. No current standard library container has both strong insertion and non-back erasure performance, so again this is a good match for `hive`.

[Reports from other fields](#) suggest that, because most developers aren't aware of containers such as this, they often end up using solutions which are sub-par for iterative performance such as `std::map` and `std::list` in order to preserve pointer validity, when most of their processing work is actually iteration-based. So, introducing this container would both create a convenient solution to these situations, as well as increasing awareness of this approach. It will ease communication across fields, as opposed to the current scenario where each field uses a similar container but each has a different name for it (object pool, bucket array, etcetera).

IV. Impact On the Standard

This is purely a library addition, requiring no changes to the language.

V. Design Decisions

Core aspects

The three core aspects of a hive from an abstract perspective are:

1. A collection of element blocks + metadata, to prevent reallocation during insertion (as opposed to a single element block).
2. A method of skipping erased elements in $O(1)$ time during iteration (as opposed to reallocating subsequent elements during erasure).
3. An erased-element location recording mechanism, to enable the re-use of memory from erased elements in subsequent insertions, which in turn increases cache locality and reduces the number of block allocations/deallocations.

Each element block houses multiple elements. The metadata about each block may or may not be allocated with the blocks themselves and could be contained in a separate structure. This metadata must include [at a minimum](#), the number of non-erased elements within each block and the block's capacity - which allows the container to know when the block is empty and needs to be removed from the sequence, and also allows iterators to judge when the end of a block has been reached, given the starting point of the block.

It should be noted that most of the data associated with the skipping mechanism and erased-element recording mechanisms should be per-element-block and independent of subsequent/previous element blocks, as otherwise you would create unacceptably variable latency [for any fields involving timing sensitivity](#). Specifically with a global data set for either, erase would likely require all data subsequent to a given element block's data to be reallocated, when an element block is removed from the iterative sequence. Insert would likewise require reallocation of all data to a larger memory space when hive capacity expanded.

In the [original](#) reference implementation (current reference implementation is [here](#)) the specific structure and mechanisms have changed many times over the course of development, however the interface to the container and its

time complexity guarantees have remained largely unchanged. So it is likely that regardless of specific implementation, it will be possible to maintain this interface without obviating future improvement.

The current reference implementation implements the 3 core aspects as follows. Information about known alternative ways to implement these is available in [the appendices](#).

1. Collection of element blocks + metadata

In the reference implementation this is essentially a doubly-linked list of 'group' structs containing (a) a dynamically-allocated element block, (b) element block metadata and (c) a dynamically-allocated skipfield. The element blocks and skipfields have a growth factor of 2. The metadata includes information necessary for an iterator to iterate over hive elements, such as that already mentioned and information useful to specific functions, such as the group's sequence order number (used for iterator comparison operations). This linked-list approach keeps the operation of removing empty element blocks from the sequence at $O(1)$ time complexity.

2. A method of skipping erased elements in $O(1)$ time during iteration

The reference implementation uses a skipfield pattern called the [low complexity jump-counting pattern](#). This encodes the length of runs of contiguous erased elements (skipblocks) into a skipfield which allows for $O(1)$ time complexity during iteration (see the paper above for details). Since there is no branching involved in iterating over the skipfield aside from end-of-block checks, it is less problematic computationally than a boolean skipfield (which has to branch for every skipfield read) in terms of CPUs which don't handle branching or branch-prediction failure efficiently (eg. Core2).

3. Erased-element location recording mechanism

The reference implementation utilizes the memory space of erased elements to form a per-element-block index-based doubly-linked [free list](#) of skipblocks, which is used during subsequent insertion. Each element block has a 'free list head' as a metadata member. The free lists are index-based rather than pointer-based in order to reduce the amount of space necessary to store the 'previous' and 'next' list links in an erased element's memory. The beginning and end of the free lists are marked using `numeric_limits<skipfield_type>::max()` in the 'previous' and 'next' indexes, respectively. If the free list head is equal to this number this means there are no erasures in that element block. Since this number is reserved that means element block capacities cannot be larger than `numeric_limits<skipfield_type>::max()` ie. 255 elements instead of 256 for 8-bit skipfield types, as otherwise the free list would be unable to address a skipblock comprised only of the last element in the block.

These per-element-block free lists are combined with a doubly-linked pointer-based intrusive list of blocks with erased elements in them, the head of which is stored as a member variable in hive. The combination of these two things allows re-use of erased element memory space in $O(1)$ time.

More information on these approaches, and alternative approaches to the 3 core aspects, is available to read in [the alt implementation appendix](#).

Iterator classes

Iterators are bidirectional in hive but also provide constant time complexity $>$, $<$, $>=$, $<=$ and $<=>$ operators for convenience (eg. in for loops when skipping over multiple elements per loop and there is a possibility of going past a pre-determined end element). This is achieved by keeping a record of the relative order of element blocks. In the reference implementation this is done by assigning a number to each memory block in its metadata. In an implementation using a vector of pointers to groups instead of a linked list, one can simply use the position of the pointers within the vector to determine this. Comparing the relative order of two iterators' blocks, then comparing the memory locations of the elements which the iterators point to (if they happen to be within the same memory block), is enough to implement all comparisons.

Iterator implementations are dependent on the approach taken to core aspects 1 and 2 as described above. The reference implementation's iterator stores a pointer to the current 'group' struct, plus a pointer to the current element and a pointer to its corresponding skipfield node. It is possible to replace the element and skipfield pointers with a single index value, but benchmarks have shown this to be slower despite the increased memory cost.

The reference implementation's ++ operation is as shown below, following the low-complexity jump-counting pattern's algorithm:

1. Add 1 to the existing element and skipfield pointers in the iterator.
2. Dereference skipfield pointer to get the value of the skipfield node, then add that value to both the skipfield pointer and the element pointer. If the node indicates an erased element, its value will be a positive integer indicating the number of nodes until the next non-erased node. If not erased it will be zero.
3. If the element pointer is now beyond the end of the element block, change the group pointer to the next group in the linked list, element pointer to the start of the next group's element block, skipfield pointer to the start of the next group's skipfield. In case there is a skipblock at the beginning of this element block, again dereference the skipfield pointer to get the value of the skipfield node and add that value to both the skipfield pointer and the element pointer. There is no need to repeat the check for end of block, because if the block were empty of elements it would've been removed.

-- operation is the same except both step 1 and 2 involve subtraction rather than addition and step 3 checks to see if the element pointer is now before the beginning of the element block instead of beyond the end of it. If it is before the beginning of the block it traverses to the back element of the previous group's element block, and subtracts the value of the back skipfield node from the element pointer and skipfield pointer.

We can see from the above that every so often iteration will involve a transition to the next/previous element block in the hive's sequence of active blocks, depending on whether we are doing ++ or --. Hence for every element block transition, 2 reads of the skipfield are necessary instead of 1.

Specific functions

- Non-member function specializations for **advance**, **prev** and **next**
(between $O(1)$ and $O(n)$)

For these functions, complexity is dependent on the state of the hive instance, position of the iterator and the amount of distance to travel, but in many cases will be less than linear, and may be constant. To explain: it is necessary in a hive to store, for each element block, both *capacity* metadata (for the purpose of iteration) and metadata about how many non-erased elements are present (ie. *size*, for the purpose of removing blocks from the iterative chain once they become empty). For this reason, intermediary blocks between the iterator's initial block and its final destination block

(if these are not the same block, and are not immediately adjacent) can be skipped rather than iterated linearly across, by using the *size* metadata.

This means that the only linear time operations are any iterations within the initial block and the final block. However if either the initial or final block have no erased elements (as determined by comparing whether the block's *capacity* and *size* metadata are equal), linear iteration can be skipped for that block and pointer/index math used instead to determine distances, reducing complexity to constant time. Finally, if the iterator points to the first element in that element block, and distance is greater-or-equal-to the block's *size*, we can treat it as an intermediary block and just skip it, subtracting *size* from the distance we want to travel. Hence the best case for this operation is constant time, the worst is linear in the distance.

- Non-member function specialization for **distance(first, last)**
(between $O(1)$ and $O(n)$)

The same considerations which apply to advance, prev and next also apply to distance - intermediary element blocks between first and last's blocks can be skipped in constant time and their *size* metadata added to the cumulative distance count, while first's block and last's block (if they are not the same block) must be linearly iterated across unless either block has no erased elements, in which case the operation becomes pointer/index math and is reduced to constant time for that block. If first and last are in the same block but are in the first and last element slots in the block, distance can again be calculated from the block's *size* metadata in constant time. If they are not in the same block but first points to the first element in it's block, the first block can be skipped and it's *size* added to the distance travelled. Likewise if last points to the last element in it's block, the last block can also be skipped and it's *size* added.

- **iterator insert/emplace**
($O(1)$ amortized)

Insertion can re-use previously-erased element memory locations when available, so position of insertion is effectively random unless no previous erasures have occurred, in which case all elements will likely be inserted linearly to the back of the container in the majority of implementations. If inserting to the back this invalidates iterators pointing to end(). It could also potentially insert before begin(), if erasures have occurred at the beginning of the container.

While it is not mandated to do so, hive implementations will generally insert into existing element blocks when able, and create a new element block only when all existing element blocks are full.

If hive is implemented as a vector of pointers to element blocks instead of a linked list of element blocks, creation of a new element block would occasionally involve expanding the pointer vector, itself $O(n)$ in the number of blocks, but this is within amortized limits since it is only occasional.

- **void insert**
($O(n)$)

For range, fill and initializer_list insertion, it is not possible to guarantee that all the elements inserted will be sequential in the hive's sequence, and so it is not considered useful to return an iterator to the first inserted element. There is a precedent for this in the various std:: map containers. Therefore these functions return void.

The same considerations regarding iterator invalidation in singular insertion above, also applies to these insertion styles.

For multiple insertions an implementation can call reserve() in advance, reducing the number of allocations necessary (whereas repeated singular insertions would generally follow the implementation's block growth factor, and possibly allocate more and smaller element blocks than necessary). This has no effect on time complexity which is still linear in the number elements inserted.

- **iterator erase(const_iterator position)**
($O(1)$ amortized)

Erasure is a simple matter of destructing the element in question and updating whatever data is associated with the erased-element skipping mechanism. No reallocation of subsequent elements is necessary and hence the process is $O(1)$. Updates to the erased-element recording and skipping mechanisms are also required to be $O(1)$.

When an element block becomes empty of non-erased elements it must be freed to the OS (or reserved for future insertions, depending on implementation) and removed from the hive's sequence of active blocks. If it were not, we would end up with non- $O(1)$ iteration, since there would be no way to predict how many empty element blocks were between the current element block being iterated over, and the next element block with non-erased elements in it.

In a linked-list-of-blocks style of implementation this removal is always $O(1)$. However if the hive were implemented as vector of pointers to element blocks, this could, depending on implementation, trigger an $O(n)$ relocation of subsequent block pointers in the vector (a smart implementation would only do this occasionally, using `erase_if` - see the [alt implementation appendix](#)). Hence this operation is $O(1)$ amortized.

Under what circumstances element blocks are reserved rather than deallocated is implementation-defined - however given that small memory blocks have low cache locality compared to larger ones, from a performance perspective it is best to only reserve the largest blocks currently allocated in the hive. In my benchmarking, reserving both the back and 2nd-to-back element blocks while ignoring the actual capacity of the blocks themselves seemed to have the most beneficial performance characteristics out of other techniques attempted.

There are three main performance advantages to retaining back blocks as opposed to just any block - the first is that these will be, under most circumstances, the largest blocks in the hive (given the growth factor). An exception to this is when splice is used, which may result in a smaller block following a larger block (implementation-dependent). The second advantage is that in situations where erasures and insertions are occurring at the back of the hive (this assumes no erased element locations in other memory blocks, which would most likely be used for the insertions) continuously and in quick succession, retaining the back block avoids large numbers of deallocations/reallocations. The third advantage is that deallocations of these larger blocks can, in part, be moved by the user to non-critical code regions via `trim_capacity()`. Though ultimately if the user wants total control of when allocations and deallocations occur they would need to use a custom allocator.

Lastly, the reason for returning an iterator is: if an erasure empties an element block of elements, the block will be deallocated or reserved - in either case, it's no longer part of the iterative sequence and an iterator pointing into it, such as `position`, can no longer be used for iteration. This is important for erasing inside a loop.

- **`iterator erase(const_iterator first, const_iterator last)`**
($O(n)$ in the number of blocks within the range, and $O(n)$ in the number of elements erased)

The same considerations for singular erasure above also apply for range-erasure. In addition, ranged erasure is $O(n)$ if elements are non-trivially-destructible. If they are trivially-destructible, we can follow similar logic to the distance specialization above. Which is to say, for the first and last element blocks in the range, if the number of elements in either block are equal to their capacity, there are no erasures in the block and we may be able to - depending on the erased-element-skipping-mechanism - simply notate a new skipblock without needing to deal with any existing skipblocks. If there are erasures in that element block, we would (implementation-dependent) likely need to identify whether the range we're erasing contains erased elements in between the non-erased elements, in order to update metadata (such as number of non-erased elements in the block) correctly.

For intermediary blocks between the first and last blocks, for trivially-destructible types we can simply deallocate or reserve these without calling the destructors of elements or dealing with the erased-element skipping/recording mechanisms for those blocks. As with distance, if the first iterator points to the first element in it's element block, the first block can be treated like an intermediary block - likewise for the last block, if the last iterator points to the last element in it's element block. Hence for trivially-destructible types, the entire operation can be linear in the number of blocks contained within the range or linear in the number of elements contained within the range, or somewhere in between.

As with singular erasure, in a vector-of-pointers-to-blocks style of implementation, there may be a need to reallocate element block pointers backward when blocks becomes empty of elements.

Lastly, specifying a return iterator for range-erase may seem pointless, as no reallocation of elements occurs in erase for hive, so the return iterator will almost always be the last const_iterator of the first, last pair. However if last was end(), the new value of end() will be returned. In this case either the user intentionally submitted end() as last, or they incremented an iterator pointing to the final element in the hive and submitted that as last. The latter is the only valid reason to return an iterator from the function, as it may occur as part of a loop which is erasing elements which ends when end() is reached. If end() is changed by the erasure, but the iterator used in the loop does not accurately reflect end()'s new value, that iterator could iterate past end() and the loop would never end.

- **void reshape(std::hive_limits block_limits)**

(between $O(1)$ and $O(n)$ in the number of elements reallocated + $O(n)$ in the number of element blocks)

This function updates the block capacity limits in the hive with user-defined ones and, if necessary, changes any active blocks which fall outside of those limits to be within the limits (and deallocates any reserved blocks outside of the limits - although an implementation could choose to allocate new reserved blocks if they wanted). A program will not compile if the function is used with non-copyable/movable types. It will invalidate pointers/iterators/references to if reallocation of elements to other element blocks occurs.

The order of elements post-reshape is not guaranteed to be stable, in order to allow for optimizations. Specifically: in the instance where a given element element block does not fit within the limits supplied, the elements within that element block could be reallocated to previously-erased element locations in other element blocks which *do* fit within the limits supplied. Or they could be reallocated to the back of the final element block, if it fits within the limits, or into reserved blocks if they fit within the limits.

If the existing current limits fit within the new user-supplied ones, no checking of block capacities is needed and the operation is $O(1)$. If they do not but existing blocks *may* fit within the limits, all blocks need to be checked, making the operation $O(n)$ in the number of blocks (both active and reserved). If any blocks containing elements don't fit within the supplied limits reallocation will occur and the operation is at worst $O(n)$ in capacity().

- **static constexpr std::hive_limits block_capacity_hard_limits() noexcept**
($O(1)$)

As opposed to block_capacity_limits() which returns the current min/max element block capacities for a given instance of hive, this allows the user to get any implementation's min/max 'hard' lower/upper limits for element block capacities ie. the limits which any user-supplied limits must fit within. For example, if an implementation's hard limit is 3 elements min, 1 million elements max, all user-supplied limits must be ≥ 3 and ≤ 1 million.

This is useful for 2 reasons:

- a. An implementation may have default block capacity limits which are different from its hard limits.
- b. A user must have a mechanism for determining what user-defined limits they can supply before supplying them to a constructor, to avoid triggering an exception. This is most important when working with projects on multiple platforms.

- **void clear()**

($O(n)$ in the number of elements)

User expectation was that clear() would erase all elements but not deallocate element blocks. Therefore all active blocks are emptied of elements and become reserved blocks. If deallocation of memory blocks is desired, a clear() call can be followed by a trim_capacity() call. For trivially-destructible types element destruction can be skipped and depending on implementation the process may be $O(1)$.

- **iterator get_iterator(const_pointer p) noexcept**
const_iterator get_iterator(const_pointer p) const noexcept
($O(n)$ in the number of active blocks)

Because hive iterators could be large, potentially storing three pieces of data - eg. pointers to: current element block, current element and current skipfield node - a program storing many links to elements within a hive may opt to dereference iterators to get pointers and store those instead of iterators, to save memory and improve performance via reduced cache use. This function reverses that process, giving an iterator which can then be used for operations such as erase. A `get_const_iterator` function was fielded as a workaround for the possibility of someone wanting to supply a non-const pointer and get a `const_iterator` back, however `as_const` fulfills this same role when supplied to `get_iterator` and doesn't require expanding the interface of hive. Likewise it was decided to use `const_pointer's` because if a user wants to supply a non-const pointer they can use `as_const`, whereas there is no meaningful equivalent process to convert `const_pointer` to pointer.

Note that this function is only guaranteed to return an iterator that corresponds to the pointer supplied - it makes no checks to see whether the element which `p` originally pointed to is the same element which `p` now points to (eg. from an ABA scenario). Resolving this problem is down to the end user and could involve having a unique id within elements or similar (more info in the [frequently-asked questions appendix](#)).

Technically, a precondition of the function is that `p` points to an element in `*this`, and does not point to an erased element, otherwise behaviour is undefined. This is due to the "[lifetime pointer zap](#)" issue ie. reading the value of a pointer to an erased element is undefined behaviour in C++. In practice this is usually non-problematic and many fields are fine with this situation. The reference implementation returns `end()` when `p` is not an element in `*this` and it is possible that other implementations may do the same. LEWG decided to remove this as an Effect of the function due to the UB mentioned.

Note 1: in order to check whether a given element is erased when an implementation is using the [low-complexity jump-counting pattern](#), the additional operations specified under "Parallel processing" in that paper must be followed.

Note 2: `get_iterator` compares pointers against the start and end memory locations of the active blocks in `*this`. There was some confusion that this would be problematic due to obscure rules in the standard which state that a given platform may allow addresses inside of a given memory block to essentially not be contiguous, at least in terms of the `std::less/std::greater/>/</etc` operators. According to Jens Maurer, these difficulties can be bypassed via hidden channels between the library implementation and the compiler.

- **`void shrink_to_fit()`**
(at worst $O(n)$ in the number of elements)

A decision had to be made as to whether this function should, in the context of hive, be allowed to reallocate elements (as `std::vector` implementations tend to do) or simply trim off reserved blocks (as `std::deque` implementations tend to do). Due to the fact that a large hive memory block could have as few as one remaining element in a large active block after a series of erasures, it makes little sense to only trim reserved blocks, so instead a `shrink_to_fit` reallocates all elements to as few active blocks as possible in order to increase cache locality during iteration and reduce memory usage. It cannot guarantee that `size() == capacity()` after the operation, because the min/max block capacity limits of `*this` may prevent that.

One potential implementation is fairly brute-force - create a new temporary hive, `reserve(size()` of original hive), copy/move all elements from the original hive into the temporary, then `operator =` && the temporary into the original. A more astute implementation might allocate a temporary array detailing the full capacity and unused capacity of each block, then use some procedure to move elements out of some blocks and into as few of the existing blocks as possible, filling up any erased element locations and/or unused space at the back of the hive and only allocating new element blocks as-necessary. The latter approach is why the order of elements post-reshape is not guaranteed to be stable.

- **`void trim_capacity()`**
`void trim_capacity(size_type n)`
($O(n)$ in the number of reserved blocks deallocated)

The `trim_capacity()` command was also introduced as a way to free reserved blocks which had been previously created via `reserve()` or transformed from active blocks to reserved blocks via `erase()`, without reallocating elements

and invalidating iterators as `shrink_to_fit()` does. The second overload was introduced as a way of allowing the user to say "I want to retain at least `n` capacity while freeing reserved blocks, so that I have room for future insertions without having to allocate again". This means the user doesn't have to know how much unused capacity is in (a) unused element memory space in the back block, (b) unused element memory space from prior erasures, or (c) reserved blocks. They just say how much they want to retain, and the implementation will free as much of the the remainder (`capacity() - n`) as possible if there are suitable reserved blocks available to deallocate.

- **`void sort()`**

($O(n \log n)$)

Although the container has unordered insertion, there may be circumstances where sorting is desired. Because hive uses bidirectional iterators, using `std::sort` or other random access sort techniques is not possible. Therefore an internal sort routine is supplied, bringing it in line with `std::list`. An implementation of the sort routine used in the reference implementation of hive can be found in a non-container-specific form [here](#) - see that page for the technique's advantages over the usual sort algorithms for non-random-access containers. An allowance is made for sort to allocate memory if necessary, so that algorithms such as this can be used. Since memory allocation is unspecified but allowed for `std::list::sort/std::forward_list::sort`, this is not necessary, just courtesy.

- **`void unique();`**

**`template <class BinaryPredicate>`
`size_type unique(BinaryPredicate binary_pred);`**

(between $O(1)$ and $O(n - 1)$)

Likewise, if a container can be sorted, `unique()` may be of use post-sort. Optimal implementation of `unique` involves calling the range-erase function where possible, as range-erase has potentially constant time depending on the state of the given blocks, as opposed to calling single-element `erase()` repeatedly which would be at worst $O(n - 1)$.

- **`void splice(hive &x)`**

`void splice(hive &&x)`

(at best $O(1)$, at worst $O(n)$ in the number of blocks in `*this` + the number of blocks in `x`)

Whether `x`'s active blocks are transferred to the beginning or end of `*this`'s sequence of active blocks, or interlaced in some way (for example to order blocks by their capacity from small to large) is implementation-defined. Better performance may be gained in some cases by allowing the source's active blocks to go to the front rather than the back, depending on how full the final active block in `x` is. This is because unused elements that are not at the back of hive's iterative sequence will need to be marked as skipped in some way, and skipping over large numbers of elements will incur a small performance disadvantage during iteration compared to skipping over a small number of elements, due to memory locality.

This function may throw in three ways - the first is a `length_error` exception if any of the capacities of `x`'s active blocks are outside of `*this`'s block capacity limits. The second is an exception if the allocators of the two hives are different. Third is a potential `bad_alloc` in the case of a [vector-of-pointers-to-blocks](#) style of implementation, where an allocation may be made if `*this`'s pointer vector isn't of sufficient capacity to accomodate the pointers to `x`'s active blocks.

For that scenario the time complexity (to expand the vector and reallocate all pointers) is linear in the number of the element blocks in `*this` + the number of active blocks in `x`. But regardless of implementation, a check needs to be made as to whether the `x`'s active blocks are within `*this`'s current block limits. This check may be $O(1)$ or $O(n)$ in the number of `x`'s active blocks depending on the values of `*this`'s and `x`'s current limits (same logic as `reshape()` above).

Final note: reserved blocks in `x` are not transferred into `*this`. This was decided by LEWG to be non-implementation defined in order to not create unexpected behaviour when moving from one implementation to another. An implementation may need to count the amount of capacity stored in reserved blocks in the two hive instances in order to correct the total capacity values of both, which may involve a traversal of reserved blocks.

VI. Technical Specification

Suggested location of hive in the standard is Sequence Containers.

Header <version> synopsis [version.syn]

```
#define __cpp_lib_hive <editor supplied value> // also in <hive>
```

General [containers.general]

Containers library summary [tab:containers.summary]

Subclause	Header
Requirements	
Sequence containers	<array>, <deque>, <forward_list>, <list>, <vector>, <hive>
Associative containers	<map>, <set>
Unordered associative containers	<unordered_map>, <unordered_set>
Container adaptors	<queue>, <stack>
Views	

Sequence containers [sequence.reqmts]

1. A sequence container organizes a finite set of objects, all of the same type, into a strictly linear arrangement. The library provides four basic kinds of sequence containers: vector, forward_list, list, and deque. In addition, array is provided as a sequence container which provides limited sequence operations because it has a fixed number of elements and hive are provided as sequence containers which provide limited sequence operations, in array's case because it has a fixed number of elements, and in hive's case because insertion order is unspecified. The library also provides container adaptors that make it easy to construct abstract data types, such as stacks or queues, out of the basic sequence container kinds (or out of other kinds of sequence containers that the user defines).

Header <hive> synopsis [hive.syn]

```
#include <initializer_list> // see [initializer.list.syn]
#include <compare> // see [compare.syn]

namespace std {
```

```

// class template hive

struct hive_limits;

template <class T, class Allocator = allocator<T>> class hive;

template<class T, class Allocator>
    void swap(hive<T, Allocator>& x, hive<T, Allocator>& y)
        noexcept(noexcept(x.swap(y)));

template<class T, class Allocator, class U>
    typename hive<T, Allocator>::size_type
        erase(hive<T, Allocator>& c, const U& value);

template<class T, class Allocator, class Predicate>
    typename hive<T, Allocator>::size_type
        erase_if(hive<T, Allocator>& c, Predicate pred);

namespace pmr {
    template <class T>
        using hive = std::hive<T, polymorphic_allocator<T>>;
}
}

```

22.3.14 Class template hive [hive]

22.3.14.1 Overview [hive.overview]

1. A hive is a sequence container that allows constant-time insert and erase operations. Insertion position is not specified, but will in most implementations typically be the back of the container when no erasures have occurred, or when erasures have occurred it can re-use existing erased element memory locations. Storage management is handled automatically and is specifically organized in multiple blocks of elements, referred to hereon as *element blocks*.
2. Element blocks which are interacted with when iterating over the sequence are referred to as *active blocks*, those which are not are referred to as *reserved blocks*. Active blocks which become empty of sequence elements [Example: via use of `erase` or `clear` - end example] are either deallocated or become reserved blocks. Reserved blocks can be allocated by the user [Example: by `reserve` - end example] for future use during insertions, at which point they become active blocks.
3. Erasures use unspecified techniques to mark erased elements as being erased, as opposed to relocating subsequent elements during erasure as is expected in a vector or deque. These elements are subsequently skipped during iteration. The same, or different unspecified techniques may be used to record the locations of erased elements, such that those locations may be reused later during insertions. All of these techniques have constant time complexity.
4. Active block capacities have an implementation-defined growth factor (which need not be integral), for example a new active block's capacity could be equal to the summed capacities of the pre-existing active blocks.
5. Limits can be placed on both the minimum and maximum element capacities of element blocks, both by users and implementations.

- (5.1) — The minimum limit shall be no larger than the maximum limit.
- (5.2) — When limits are not specified by a user during construction, the implementation's default limits are used.
- (5.3) — The default limits of an implementation are not guaranteed to be the same as the minimum and maximum possible capacities for an implementation's element blocks [Note 1: To allow latitude for both implementation-specific and user-directed optimization. - end note]. The latter are defined as *hard limits*.
- (5.4) — If user-specified limits are not within hard limits, or if the specified minimum limit is greater than the specified maximum limit, behavior is undefined.
- (5.5) — After construction a hive's limits can be changed by the user via reshape.
- (5.6) — Limits are copied between hives during copy construction, move construction, `operator = (hive &&)` and `swap`.
- (5.7) — An element block is said to be *within the bounds* of a pair of minimum/maximum limits when it's capacity is $\geq$ the minimum limit and $\leq$ the maximum limit.
6. A hive conforms to the requirements for Containers ([container.reqmts]), with the exception of operators `==` and `!=`. A hive also meets the requirements of a reversible container ([container.rev.reqmts]), of an allocator-aware container ([container.alloc.reqmts]), and some of the requirements of a sequence container, including several of the optional sequence container requirements ([sequence.reqmts]). Descriptions are provided here only for operations on hive that are not described in that table or for operations where there is additional semantic information.
7. Hive iterators meet the Cpp17BidirectionalIterator requirements but also provide relational operators `<`, `<=`, `>`, `>=` and `<=>` which compare the relative ordering of two iterators in the sequence.

```
namespace std {

struct hive_limits
{
    size_t min;
    size_t max;
    constexpr hive_limits(size_t minimum, size_t maximum) noexcept :
min(minimum), max(maximum) {}
};

template <class T, class Allocator = allocator<T>>
class hive {
private:
    hive_limits current_limits = implementation-defined; // exposition only

public:

    // types
    using value_type = T;
    using allocator_type = Allocator;
    using pointer = typename allocator_traits<Allocator>::pointer;
    using const_pointer = typename
allocator_traits<Allocator>::const_pointer;
    using reference = value_type&;
    using const_reference = const value_type&;
```

```

    using size_type = implementation-defined; // see [container.requirements]
    using difference_type = implementation-defined; // see
[container.requirements]
    using iterator = implementation-defined; // see [container.requirements]
    using const_iterator = implementation-defined; // see
[container.requirements]
    using reverse_iterator = std::reverse_iterator<iterator>; // see
[container.requirements]
    using const_reverse_iterator = std::reverse_iterator<const_iterator>; //
see [container.requirements]

constexpr hive() noexcept(noexcept(Allocator())) : hive(Allocator()) { }
explicit hive(const Allocator&) noexcept;
explicit hive(hive_limits block_limits) : hive(block_limits, Allocator())
{ }
    hive(hive_limits block_limits, const Allocator&);
    explicit hive(size_type n, const Allocator& = Allocator());
    hive(size_type n, hive_limits block_limits, const Allocator& =
Allocator());
    hive(size_type n, const T& value, const Allocator& = Allocator());
    hive(size_type n, const T& value, hive_limits block_limits, const
Allocator& = Allocator());
    template<class InputIterator>
        hive(InputIterator first, InputIterator last, const Allocator& =
Allocator());
    template<class InputIterator>
        hive(InputIterator first, InputIterator last, hive_limits block_limits,
const Allocator& = Allocator());
    template<container-compatible-range<T> R>
        hive(from_range_t, R&& rg, const Allocator& = Allocator());
    template<container-compatible-range<T> R>
        hive(from_range_t, R&& rg, hive_limits block_limits, const Allocator& =
Allocator());
    hive(const hive& x);
    hive(hive&&) noexcept;
    hive(const hive&, const type_identity_t<Allocator>&);
    hive(hive&&, const type_identity_t<Allocator>&);
    hive(initializer_list<T> il, const Allocator& = Allocator());
    hive(initializer_list<T> il, hive_limits block_limits, const Allocator& =
Allocator());

    ~hive();
    hive& operator=(const hive& x);
    hive& operator=(hive&& x)
noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignmen
t::value || allocator_traits<Allocator>::is_always_equal::value);
    hive& operator=(initializer_list<T>);
    template<class InputIterator>
        void assign(InputIterator first, InputIterator last);
    template<container-compatible-range <T> R>
        void assign_range(R&& rg);
    void assign(size_type n, const T& t);
    void assign(initializer_list<T>);
    allocator_type get_allocator() const noexcept;

```

```

// iterators
iterator          begin() noexcept;
const_iterator    begin() const noexcept;
iterator          end() noexcept;
const_iterator    end() const noexcept;
reverse_iterator  rbegin() noexcept;
const_reverse_iterator rbegin() const noexcept;
reverse_iterator  rend() noexcept;
const_reverse_iterator rend() const noexcept;

const_iterator    cbegin() const noexcept;
const_iterator    cend() const noexcept;
const_reverse_iterator crbegin() const noexcept;
const_reverse_iterator crend() const noexcept;

// capacity
[[nodiscard]] bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;
size_type capacity() const noexcept;
void reserve(size_type n);
void shrink_to_fit();
void trim_capacity() noexcept;
void trim_capacity(size_type n) noexcept;

// modifiers
template <class... Args> iterator emplace(Args&&... args);
iterator insert(const T& x);
iterator insert(T&& x);
void insert(size_type n, const T& x);
template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
template<container-compatible-range <T> R>
    void insert_range(R&& rg);
void insert(initializer_list<T> il);

iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void swap(hive&)
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
allocator_traits<Allocator>::is_always_equal::value);
void clear() noexcept;

// hive operations
void splice(hive& x);
void splice(hive&& x);
size_type unique();
template<class BinaryPredicate>
    size_type unique(BinaryPredicate binary_pred);

```

```

constexpr hive_limits block_capacity_limits() const noexcept;
static constexpr hive_limits block_capacity_hard_limits() noexcept;
void reshape(hive_limits block_limits);

iterator get_iterator(const_pointer p) noexcept;
const_iterator get_iterator(const_pointer p) const noexcept;

void sort();
template <class Compare> void sort(Compare comp);
}

template<class InputIterator, class Allocator = allocator<iter-value-type
<InputIterator>>
    hive(InputIterator, InputIterator, Allocator = Allocator())
        -> hive<iter-value-type <InputIterator>, Allocator>;

template<class InputIterator, class Allocator = allocator<iter-value-type
<InputIterator>>
    hive(InputIterator, InputIterator, hive_limits block_limits, Allocator =
Allocator())
        -> hive<iter-value-type <InputIterator>, block_limits, Allocator>;

template<ranges::input_range R, class Allocator =
allocator<ranges::range_value_t<R>>>
    hive(from_range_t, R&&, Allocator = Allocator())
        -> hive<ranges::range_value_t<R>, Allocator>;

template<ranges::input_range R, class Allocator =
allocator<ranges::range_value_t<R>>>
    hive(from_range_t, R&&, hive_limits block_limits, Allocator =
Allocator())
        -> hive<ranges::range_value_t<R>, block_limits, Allocator>;
}

```

22.3.14.2 Constructors, copy, and assignment [hive.cons]

explicit hive(const Allocator&) noexcept;

1. Effects: Constructs an empty hive, using the specified allocator.
2. Complexity: Constant.

hive(hive_limits block_limits, const Allocator&);

3. Effects: Constructs an empty hive, using the specified allocator. Initializes current - limits with block_limits.
4. Complexity: Constant.


```
explicit hive(size_type n, const Allocator& = Allocator());  
hive(size_type n, hive_limits block_limits, const Allocator& = Allocator());
```

- 5. Preconditions: T is *Cpp17DefaultInsertable* into *this.
- 6. Effects: Constructs a hive with n default-inserted elements, using the specified allocator. If the second overload is called, also initializes current-limits with block_limits.
- 7. Complexity: Linear in n.

```
hive(size_type n, const T& value, const Allocator& = Allocator());  
hive(size_type n, const T& value, hive_limits block_limits, const Allocator& =  
Allocator());
```

- 8. Preconditions: T is *Cpp17CopyInsertable* into *this.
- 9. Effects: Constructs a hive with n copies of value, using the specified allocator. If the second overload is called, also initializes current-limits with block_limits.
- 10. Complexity: Linear in n.

```
template<class InputIterator>  
    hive(InputIterator first, InputIterator last, const Allocator& = Allocator());  
template<class InputIterator>  
    hive(InputIterator first, InputIterator last, hive_limits block_limits, const  
Allocator& = Allocator());
```

- 11. Effects: Constructs a hive equal to the range [first, last), using the specified allocator. If the second overload is called, also initializes current-limits with block_limits.
- 12. Complexity: Linear in distance(first, last).

```
template<container-compatible-range<T> R>  
    hive(from_range_t, R&& rg, const Allocator& = Allocator());  
template<container-compatible-range<T> R>  
    hive(from_range_t, R&& rg, hive_limits block_limits, const Allocator& = Allocator());
```

- 13. Effects: Constructs a hive object with the elements of the range rg, using the specified allocator. If the second overload is called, also initializes current-limits with block_limits.
- 14. Complexity: Linear in ranges::distance(rg).

```
hive(initializer_list<T> il, const Allocator& = Allocator());  
hive(initializer_list<T> il, hive_limits block_limits, const Allocator& = Allocator());
```

- 16. Preconditions: T is *Cpp17CopyInsertable* into *this.
- 17. Effects: Constructs a hive object with the elements of il, using the specified allocator. If the second overload is called, also initializes current-limits with block_limits.
- 18. Complexity: Linear in il.size().

22.3.14.3 Capacity [hive.capacity]

size_type capacity() const noexcept;

1. Returns: The total number of elements that the hive can hold without requiring allocation of more element blocks.
2. Complexity: Constant.

void reserve(size_type n);

3. Effects: A directive that informs a hive of a planned change in size, so that it can manage the storage allocation accordingly. Does not cause reallocation of elements. Iterators to elements in **this* remain valid. If $n \leq \text{capacity}()$ there are no effects.
4. Complexity: It does not change the size of the sequence and takes at most linear time in the number of reserved blocks allocated.
5. Throws: `length_error` if $n > \text{max_size}()$ [numbered_note](#).
6. Postconditions: $\text{capacity}() \geq n$ is true.

[numbered_note](#)) `reserve()` uses `Allocator::allocate()` which can throw an appropriate exception.

void shrink_to_fit();

7. Preconditions: T is *Cpp17MoveInsertable* into **this*.
8. Effects: `shrink_to_fit` is a non-binding request to reduce `capacity()` to be closer to `size()`.
[Note: The request is non-binding to allow latitude for implementation-specific optimizations. - end note]
It does not increase `capacity()`, but may reduce `capacity()`. It may reallocate elements. If `capacity()` is already equal to `size()` there are no effects. If an exception is thrown other than by the move constructor of a non-*Cpp17CopyInsertable* T , the only effects are that `capacity()` may be reduced and reallocation may occur.
9. Complexity: If reallocation happens, linear in the size of the sequence.
10. Throws: If reallocation to new element blocks occurs, uses `Allocator::allocate()` which can throw an appropriate exception.
11. Remarks: If reallocation occurs, the order of the elements in **this* may change. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator.
[Note: If no reallocation happens, they remain valid. - end note]

void trim_capacity() noexcept;

void trim_capacity(size_type n) noexcept;

12. Effects: For the first overload, all reserved blocks are deallocated, and `capacity()` is reduced accordingly. For the second overload, `capacity()` will be reduced to no less than n .
13. Complexity: Linear in the number of reserved blocks deallocated.
14. Remarks: Does not reallocate elements and no iterators or references to elements in **this* are invalidated.

22.3.14.4 Modifiers [hive.modifiers]

```
template <class... Args>
    iterator emplace(Args&&... args);
iterator insert(const T& x);
iterator insert(T&& x);
void insert(size_type n, const T& x);
template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
template<container-compatible-range <T> R>
    void insert_range(R&& rg);
void insert(initializer_list<T> il);
```

1. Complexity: Linear in the number of elements inserted. The constructor of T is called exactly once for each element inserted.
2. Remarks: For all functions, invalidates the past-the-end iterator.

```
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
```

3. Effects: Invalidates iterators and references to the erased element. An erase operation that erases the last element of *this also invalidates the past-the-end iterator.
4. Complexity: The number of calls to the destructor of T is the same as the number of elements erased. Additionally if any active blocks become empty of elements as a result of the function call, at worst linear in the number of element blocks.

```
void swap(hive& x)
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
allocator_traits<Allocator>::is_always_equal::value);
```

5. Effects: Exchanges the contents and capacity() of *this with that of x.
6. Complexity: Constant.

22.3.14.5 Operations [hive.operations]

1. In this subclause, arguments for a template parameter named Predicate or BinaryPredicate shall meet the corresponding requirements in [algorithms.requirements]. The semantics of $i + n$ and $i - n$, where i is an iterator into the hive and n is an integer, are the same as those of $\text{next}(i, n)$ and $\text{prev}(i, n)$, respectively. For sort , the definitions and requirements in [alg.sorting] apply.

```
void splice(hive &x);
void splice(hive &&x);
```

2. Preconditions: `addressof(x) != this` is true. `get_allocator() == x.get_allocator()` is true.
3. Effects: Inserts the contents of `x` into `*this` and `x` becomes empty. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements shall continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.
4. Complexity: Linear in the sum of all element blocks in `x` plus all element blocks in `*this`.
5. Throws: `length_error` if any of `x`'s active blocks are not within the bounds of `this->current_limits`.
6. Remarks: Reserved blocks in `x` are not transferred into `*this`. Invalidates the past-the-end iterator for both `x` and `*this`.

```
size_type unique();
template<class BinaryPredicate>
    size_type unique(BinaryPredicate binary_pred);
```

7. Let `binary_pred` be `equal_to< >{} for the first overload.`
8. Preconditions: `binary_pred` is an equivalence relation.
9. Effects: Erases all but the first element from every consecutive group of equivalent elements. That is, for a nonempty hive, erases all elements referred to by the iterator `i` in the range `[begin() + 1, end())` for which `binary_pred(*i, *(i - 1))` is true. Invalidates only the iterators and references to the erased elements.
10. Returns: The number of elements erased.
11. Throws: Nothing unless an exception is thrown by the predicate.
12. Complexity: If `empty()` is false, exactly `size() - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

```
constexpr hive_limits block_capacity_limits() const noexcept;
```

13. Effects: Returns `current_limits`.
14. Complexity: Constant.

```
static constexpr hive_limits block_capacity_hard_limits() noexcept;
```

15. Returns: A `hive_limits` struct with the `min` and `max` members set to the implementation's hard limits.
16. Complexity: Constant.

```
void reshape(hive_limits block_limits);
```

17. Preconditions: `T` is *Cpp17MoveInsertable* into `*this`.
18. Effects: Assigns `block_limits` to `current_limits`. If any active blocks in `*this` are not within the bounds of `block_limits`, elements within those active blocks are reallocated to new or existing element blocks which are within the bounds. Subsequently any element blocks in `*this` which are not within the bounds of `block_limits` are deallocated. If an exception is thrown other than by the move constructor of a non-*Cpp17CopyInsertable* `T`, the only effects are that reallocation may occur and `current_limits` may change to a value other than `block_limits`.

19. Complexity: Linear in the number of element blocks in `*this`. If reallocation occurs, also linear in the number of elements reallocated.
20. Throws: If reallocation to new element blocks occurs, uses `Allocator::allocate()` which can throw an appropriate exception.
21. Remarks: This operation may change `capacity()`. If reallocation occurs, the order of the elements in `*this` may change. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator.
[Note: If no reallocation happens, they remain valid. - end note]

```
iterator get_iterator(const_pointer p) noexcept;
const_iterator get_iterator(const_pointer p) const noexcept;
```

22. Preconditions: `p` points to an element in `*this`.
23. Complexity: Linear in the number of active blocks in `*this`.
24. Returns: An iterator or `const_iterator` pointing to the same element as `p`.

```
void sort();
template <class Compare>
    void sort(Compare comp);
```

25. Preconditions: `T` is *Cpp17MoveInsertable* and *Cpp17MoveAssignable* into `*this`. lvalues of type `T` are swappable.
26. Effects: Sorts the hive according to the operator `<` or the `comp` function object. If an exception is thrown, the order of the elements in `*this` is unspecified. Iterators and references to elements may be invalidated.
27. Complexity: $N \log N$ comparisons, where $N == \text{size}()$.
28. Remarks: May allocate [numbered_note](#).
[Note: Not required to be stable ([algorithm.stable]) - end note]

`numbered_note` `sort()` uses `Allocator::allocate()` which can throw an appropriate exception.

22.3.14.6 Erasure [hive.erase]

```
template<class T, class Allocator, class U>
    typename hive<T, Allocator>::size_type
        erase(hive<T, Allocator>& c, const U& value);
```

1. Effects: Equivalent to:

```
return erase_if(c, [&](auto& elem) { return elem == value; });
```

```
template<class T, class Allocator, class Predicate>
```

```
typename hive<T, Allocator>::size_type
erase_if(hive<T, Allocator>& c, Predicate pred);
```

2. Effects: Equivalent to:

```
auto original_size = c.size();
for (auto i = c.begin(), last = c.end(); i != last; ) {
    if (pred(*i)) {
        i = c.erase(i);
    } else {
        ++i;
    }
}
return original_size - c.size();
```

VII. Acknowledgments

Matt would like to thank: Glen Fernandes and Ion Gaztanaga for restructuring advice, Robert Ramey for documentation advice, various Boost and SG14/LEWG members for support, critiques and corrections, Baptiste Wicht for teaching me how to construct decent benchmarks, Jonathan Wakely, Sean Middleditch, Jens Maurer (very nearly a co-author at this point really), Tim Song, Patrice Roy and Guy Davidson for standards-compliance advice and critiques, support, representation at meetings and bug reports, Henry Miller for getting me to clarify why the free list approach to memory location reuse is the most appropriate, Ville Voutilainen and Gašper Ažman for help with the colony/hive rename paper, Ben Craig for his critique of the tech spec, that ex-Lionhead guy for annoying me enough to force me to implement the original skipfield pattern, Jon Blow for some initial advice and Mike Acton for some influence, the community at large for giving me feedback and bug reports on the reference implementation. Also Nico Josuttis for doing such a great job in terms of explaining the general format of the structure to the committee.

Dedicated to Melodie.

VIII. Appendices

Appendix A - Basic usage examples

Using [plf::hive reference implementation](#).

```
#include <iostream>
#include <numeric>
#include "plf_hive.h"

int main(int argc, char **argv)
{
    plf::hive<int> i_hive;
```

```

// Insert 100 ints:
for (int i = 0; i != 100; ++i)
{
    i_hive.insert(i);
}

// Erase half of them:
for (plf::hive<int>::iterator it = i_hive.begin(); it != i_hive.end(); ++it)
{
    it = i_hive.erase(it);
}

std::cout << "Total: " << std::accumulate(i_hive.begin(), i_hive.end(), 0) <<
std::endl;
std::cin.get();
return 0;
}

```

Example demonstrating pointer stability

```

#include <iostream>
#include "plf_hive.h"

int main(int argc, char **argv)
{
    plf::hive<int> i_hive;
    plf::hive<int>::iterator it;
    plf::hive<int *> p_hive;
    plf::hive<int *>::iterator p_it;

    // Insert 100 ints to i_hive and pointers to those ints to p_hive:
    for (int i = 0; i != 100; ++i)
    {
        it = i_hive.insert(i);
        p_hive.insert(&(*it));
    }

    // Erase half of the ints:
    for (it = i_hive.begin(); it != i_hive.end(); ++it)
    {
        it = i_hive.erase(it);
    }

    // Erase half of the int pointers:
    for (p_it = p_hive.begin(); p_it != p_hive.end(); ++p_it)
    {
        p_it = p_hive.erase(p_it);
    }

    // Total the remaining ints via the pointer hive (pointers will still be valid
    even after insertions and erasures):
    int total = 0;

```

```

    for (p_it = p_hive.begin(); p_it != p_hive.end(); ++p_it)
    {
        total +=>(*p_it);
    }

    std::cout << "Total: " << total << std::endl;

    if (total == 2500)
    {
        std::cout << "Pointers still valid!" << std::endl;
    }

    std::cin.get();
    return 0;
}

```

Appendix B - Original reference implementation benchmarks, differences from current reference implementation, and links

Benchmark results for plf::colony (performance and majority of code is identical to std::hive reference implementation) under GCC 9.2 on an Intel Xeon E3-1241 (Haswell 2014) are [here](#).

Old benchmark results for an earlier version of colony under MSVC 2015 update 3, on an Intel Xeon E3-1241 (Haswell 2014) are [here](#). There is no commentary for the MSVC results.

Even older benchmark results for an even earlier version of colony under GCC 5.1 on an Intel E8500 (Core2 2008) are [here](#).

This proposal and its [reference implementation](#), and the [original reference implementation](#), have several differences; one is that the original was named 'colony' (as in: a human, ant or bird colony), and that name has been retained for userbase purposes but also for differentiation. Other differences between hive and colony as of the time of writing follow:

- C++20 and above only.
- ==, != and <=> container operators removed, as these were considered either too slow as order-inspecific operators, or too confusing as order-specific operators (for an unordered-insert container)
- priority template parameter removed as was not considered useful enough to maintain in standard.
- data() function removed, as may cause too much implementation specificity.
- reset() function removed (use clear() followed by trim_capacity() instead) - simply to cut down on the user interface.
- memory() removed as this was considered to be not in-keeping with previous container practice.
- No support for user-specified internal sort routines to be used by member function sort().
- advance, prev, next and distance are std:: overloads, rather than hidden friends selected via ADL. This is only possible under C++20 and above. std::distance overload doesn't support negative distances, the two iterators must be swapped prior to distance call if first is > last.
- The following public functions are not present: is_active, default_min_block_capacity, default_max_block_capacity, default_block_capacity_limits, block_metadata_memory, block_allocation_amount, max_elements_per_allocation.

Other differences may appear over time.

Appendix C - Frequently Asked Questions

1. Where is it worth using a hive in place of other std:: containers?

See the [guide to container selection appendix](#) for a more intensive answer to this question, however for a brief overview, it is worthwhile for performance reasons in situations where the order of container elements is not important and:

- a. Insertion order is unimportant
- b. Insertions and erasures to the container occur frequently in performance-critical code, *and*
- c. Links to non-erased container elements may not be invalidated by insertion or erasure.

Under these circumstances a hive will generally out-perform other std:: containers. In addition, because it never invalidates pointer references to container elements (except when the element being pointed to has been previously erased) it may make many programming tasks involving inter-relating structures in an object-oriented or modular environment much faster, and should be considered in those situations.

Some ideal situations to use a hive: cellular/atomic simulation, persistent octrees/quadtrees, game entities or destructible-objects in a video game, particle physics, anywhere where objects are being created and destroyed continuously. Also, anywhere where a vector of pointers to dynamically-allocated objects or a std::list would typically end up being used in order to preserve pointer stability, but where order is unimportant.

2. Is it similar to a deque?

A deque is reasonably dissimilar to a hive - being a double-ended queue, it requires a different internal framework. In addition, being a random-access container, having a growth factor for element blocks in a deque is problematic (though not impossible). deque and hive have no comparable performance characteristics except for insertion (assuming a good deque implementation). Deque erasure performance can vary substantially depending on implementation, but is generally similar to vector erasure performance. A deque invalidates pointers to subsequent container elements when erasing elements, which a hive does not, and guarantees ordered insertion.

3. How does a slot map compare to a hive?

Both a [slot map](#) and a hive attempt to create a container where there is reasonable insertion/erasure/iteration performance while maintaining stable links from external objects to elements within the container. In the case of hive this is done with pointers/iterators, in the case of a slot map this is done with keys, which are separate to iterators (which do not stay valid post-erasure/insertion in slot maps). Each approach has some advantages, but the hive approach has more, in my view.

If you use a slot map your external object also needs to store a link to the slot map in order to access a element from it's stored key (or a higher-level object accessing the external object needs to store such a link). This prevents effective use of splicing, since in a post-spliced slot map the external object will be referring to an element which is now in another slot map instance. With a hive there is no need for the external object to have knowledge of the hive instance, as pointers are sufficient to access the elements and maintain validity.

One upside of the slot map approach is if you make a duplicate of a slot map + a duplicate of an external object which accesses it's elements via keys, and send these to a secondary thread, little work need be done - all the keys stored in the external object duplicate will work with the copied slot map, all you need to do is update the external object to use the duplicate slot map, instead of the original.

If you wish to do this with a hive, pointers to it's elements in the duplicate external object will of course not point into the duplicate hive, so any external objects you copy to the secondary thread, which you intend to access the hive, will need their pointers to it's elements re-written. The easiest way to do this is by finding the indexes of the elements in the original hive instance via `size_type original_index = std::distance(hive.begin(), hive.get_iterator(pointer));`, then using `external_object.pointers[original_index] = &*(std::next(hive2.begin(), original_index));` to write the new pointer values. Since `std::distance/std::next/etc` can be overloaded to be very quick for hive, this will likely not take too much time, but it is an inconvenience.

While I haven't done any benchmarks comparing hive performance to a slot map, I have done extensive benchmarks vs a [packed array](#) implementation, which is arguably simpler than a slot map, and hive is faster. From the structure of a slot map it's obvious that slot maps are slower for insertion, erasure and for referencing elements within the slot map via external objects. This is because of the intermediary interfaces of the key resolution array and generation counters which need to be accessed and updated. Slot maps are however faster for iteration since all their data is stored contiguously and iteration does not use the keys/counters. In addition the contiguous storage means using a slot map with SIMD is more straight-forward.

Slot maps use more memory due to the keys/counters. Ignoring element block metadata, a hive implementation can use as little as 2 extra bits of metadata per element (current reference implementation is generally between 10 and 16 bits for performance reasons), but a slot map will typically use 128 bits per element (or 64 on a 32-bit system). This will also lower performance due to higher pressure on the cache and the increased numbers of allocations/deallocations.

4. What are the thread-safe guarantees?

Unlike a `std::vector`, a hive can be read from and inserted-into/erased-from at the same time (provided the erased element is not the same as the element being read), however it cannot be iterated over and inserted-into/erased-from to at the same time. If we look at a (non-concurrent implementation of) `std::vector`'s thread-safe matrix to see which basic operations can occur at the same time, it reads as follows (please note `push_back()` is the same as insertion in this regard due to reallocation when `size == capacity`):

std::vector	Insertion	Erasure	Iteration	Read
Insertion	No	No	No	No
Erasure	No	No	No	No
Iteration	No	No	Yes	Yes
Read	No	No	Yes	Yes

In other words, multiple reads can happen simultaneously, but the potential reallocation and pointer/iterator invalidation caused by insertion/push_back and erasure means those operations cannot occur at the same time as anything else. pop_back() is slightly different in that it doesn't cause reallocation, so pop_back()'s and reads can occur at the same time provided you're not reading from the back(). Likewise a swap-and-pop operation can occur at the same time as reading if neither the erased location nor back() is what's being read.

Hive on the other hand does not invalidate pointers/iterators to non-erased elements during insertion and erasure, resulting in the following matrix:

hive	Insertion	Erasure	Iteration	Read
Insertion	No	No	No	Yes
Erasure	No	No	No	Mostly*
Iteration	No	No	Yes	Yes
Read	Yes	Mostly*	Yes	Yes

* Erasures will not invalidate iterators unless the iterator points to the erased element.

In other words, reads may occur at the same time as insertions and erasures (provided that the element being erased is not the element being read), multiple reads and iterations may occur at the same time, but iterations may not occur at the same time as an erasure or insertion, as either of these may change the state of the skipfield which is being iterated over, if a skipfield is used in the implementation. Note that iterators pointing to end() may be invalidated by insertion.

So, hive could be considered more inherently thread-safe than a (non-concurrent implementation of) std::vector, but still has many areas which require mutexes or atomics to navigate in a multithreaded environment.

5. What is hive's Abstract Data Type (ADT)?

Though I am happy to be proven wrong I suspect hives/colonies/bucket arrays are their own abstract data type. Some have suggested its ADT is of type bag, I would somewhat dispute this as it does not have typical bag functionality such as [searching based on value](#) (you can use std::find but it's O(n)) and adding this functionality would slow down other performance characteristics. [Multisets/bags](#) are also not sortable (by means other than automatically by key value). Hive does not utilize key values, is sortable, and does not provide the sort of functionality frequently associated with a bag (e.g. counting the number of times a specific value occurs).

6. Why must active blocks be removed from the iterative sequence when empty?

Two reasons:

- Standards compliance: if active blocks aren't removed then ++ and -- iterator operations become O(n) in the number of active blocks, making them non-compliant with the C++ standard. At the moment they are O(1); in the reference implementation this constitutes typically one update for both skipfield and element pointers, two if a skipfield jump takes the iterator beyond the bounds of the current block and into the next block. But if empty blocks are allowed, there could be any number of empty blocks between the current element and the next one with elements in it. Essentially you get the same scenario as you do when iterating over a boolean skipfield.

- b. Performance: iterating over empty blocks is slower than them not being present obviously. But also, if you have to allow for empty blocks while iterating, then you have to introduce a branching loop in every iteration operation, which increases cache misses and code size. The strategy of removing blocks when they become empty also statistically removes (assuming a randomized erasure pattern) smaller blocks from the hive before larger blocks, which has a net result of improving iteration, because with a larger block, more iterations within the block can occur before the end-of-block condition is reached and a jump to the next block (and subsequent cache miss) occurs.

7. Why not turn All emptied active blocks into reserved blocks for future use during erasure, or None, rather than leaving the decision implementation-defined?

Future implementations may find better strategies, and it is best not to overly constraint implementation. For the reasons described in the *Design Decisions->Specific Functions* section on `erase()`, retaining the current two back blocks has performance and latency benefits. Therefore reserving no active blocks is non-optimal. Meanwhile, reserving All active blocks is bad for performance as many small active blocks will be reserved, which decreases iterative performance due to lower cache locality. If a user wants more fine-grained control over memory retention they may use an allocator.

8. Why is there no default constructor for `hive_limits`?

The user must obtain the block capacity hard limits of the implementation (via `block_capacity_hard_limits()`) prior to supplying their own limits as part of a constructor or `reshape()`, so that they do not trigger undefined behavior by supplying limits which are outside of the hard limits. Hence it was perceived by LEWG that there would be no reason for a `hive_limits` struct to ever be used with non-user-supplied values eg. zero.

9. Element block capacities - what are they based on, how do they expand?

There are 'hard' capacity limits, 'default' capacity limits, 'current' limits and user-defined capacity limits. Current limits are whatever the current min/max capacity limits are in a given instance of hive. Default limits are what a hive is instantiated with if user-defined capacity limits are not supplied. Both default limits and user-defined limits are not allowed to go outside of an implementation's hard limits, which represent a fixed upper and lower limit. New element blocks have a implementation-defined growth factor, so will expand up to the current max limit.

While implementations are free to chose their own limits and strategies, in the reference implementation element block sizes start from either the dynamically-defined default minimum size (8 elements, larger if the type stored is small) or an amount defined by the user (with a minimum of 3 elements, as less than 3 elements is pretty much a linked list).

Subsequent block capacities in the reference implementation increase the *total capacity* of the hive by a factor of 2 (so, 1st block 8 elements, 2nd 8 elements, 3rd 16 elements, 4th 32 elements etcetera) until the current maximum block size is reached. The default maximum block size in the reference implementation is 255 (if the type sizeof is < 10 bytes) or 8192. These values are based on multiple benchmark comparisons between different maximum block capacities, with different sized types. For larger-than-10-byte types the skipfield bitdepth is (at least) 16 so the maximum capacity 'hard' limit would be 65535 elements in that context, for < 10-byte types the skipfield bitdepth is (at least) 8, making the maximum capacity hard limit 255. Larger capacities do not necessarily perform better because, given a randomized erasure pattern, a larger block may statistically retain more erased elements ie.

empty space before it runs out of elements entirely and is removed from the sequence, and therefore can create slowdown during iteration due to low locality.

10. What are user-defined element block minimum and maximum capacities good for?

See the summary in paper P2857R0 which goes into this.

11. Why are `hive_limits` specified in constructors and not relegated to a secondary function?

- a. They have always been required in range/fill constructors for the reason that otherwise the user must construct, call reshape and then call range-assign/insert. This is slower and more cumbersome for users.
- b. They were not originally in the default constructors due to creating overload ambiguity with the fill constructors, but users have asked for this feature since 2016. One reason for the request was consistency. Another was usage with non-movable/copyable types, which cannot be used with reshape() as it must be able to reallocate elements when the existing blocks do not fit within the user-supplied range, and will throw when it cannot do so (either due to lack of memory or some other problem). The non-noexcept status of this function was also an issue for some users.
- c. In 2020 the issue was discussed [in SG14](#) and Jens Maurer [suggested](#) using an external struct to make the constructor calls unambiguous. This has been the ongoing solution. It meets the needs of those using non-movable/copyable types and provides an exception-free way for users to specify block limits (provided they check their limits using `block_capacity_hard_limits()` first).
- d. Block capacity limits are something which users have repeatedly been thankful for. They are needed and their positive characteristics are discussed in P2857R0. As a side-note, it is of annoyance to many users that similar functionality was never specified for deque.

12. Can a hive be used with SIMD instructions?

Yes if you're careful, no if you're not.

On platforms which support scatter and gather operations via hardware (e.g. AVX512) you can use hive with SIMD as much as you want, using gather to load elements from disparate or sequential locations, directly into a SIMD register, in parallel. Then use scatter to push the post-SIMD-process values back to their original locations.

In situations where gather and scatter operations are too expensive or which require elements to be contiguous in memory for SIMD processing, it is more complicated. When you have a bunch of erasures in a hive, there's no guarantee that your objects will be contiguous in memory, even though they are sequential during iteration, as there may be erased elements in between them. Some of them may also be in different element blocks to each other. In these situations if you want to use SIMD with hive, you must do the following:

- Set your minimum and maximum group sizes to multiples of the width of your target processor's SIMD instruction size. If it supports 8 elements at once, set the group sizes to multiples of 8.
- Either never erase from the hive, or:
 1. Shrink-to-fit after you erase (will invalidate all pointers to elements within the hive).

2. Only erase from the back or front of the hive, and only erase elements in multiples of the width of your SIMD instruction e.g. 8 consecutive elements at once. This will ensure that the element boundaries line up with the width of the SIMD instruction, provided you've set your min/max block sizes as above.

Generally if you want to use SIMD without gather/scatter, it's easier to use a vector or array.

13. Why were container operators ==, != and <=> removed?

Since this is a container where insertion position is unspecified, situations such as the following may occur:

```
hive<int> t = {1, 2, 3, 4, 5}, t2 = {6, 1, 2, 3, 4};  
t2.erase(t2.begin());  
t2.insert(5);
```

In this case it is implementation-defined as to whether or not `t == t2` because insertion position is not specified, if the `==` operator is order-sensitive.

If the `==` operator is order-insensitive, there is only one reasonable way to compare the two containers, which is with `is_permutation`. `is_permutation` has a worst-case time complexity of $O(n^2)$, which, while in keeping with how other unordered containers are implemented, was considered to be out of place for `hive`, which is a container where performance and consistent latency are a focus and most operations are $O(1)$ as a result. While there are order-insensitive comparison operations which can be done in $O(n \log n)$ time, these allocate, which again was considered inappropriate for a `==` operator.

In light of this the bulk of SG14/LEWG considered it more appropriate to remove the `==`, `!=` and `<=>` operators entirely, as these were unlikely to be used significantly with `hive` anyway. This gives the user the option of using `is_permutation` if they want an order-insensitive comparison, or `std::equal` if they want an order-sensitive comparison. In either case it removes ambiguity about what kind of operation they are expecting and the time complexity associated with it.

14. What hive functions can potentially change sequence order?

Asides from functions which erase existing elements like `clear`, `erase`, `~hive` and the `=` operators, the following may also change ordering: `insert/emplace` (as position is undefined), `reshape`, `shrink_to_fit`, `splice`.

15. Why was `memory()` removed?

This was a convenience function to allow programmers to find a `hive`'s current memory usage without using a debugger or profiler, however this was considered out of keeping with current standard practice ie. `unordered_map` also uses a lot of additional memory but we don't provide such a function. In addition, the context where it would've been useful in realtime (ie. determining whether or not it's worth calling `trim_capacity()` or `shrink_to_fit()`), is better approached by comparing `size()` to `capacity()`.

16. Why was the `Priority` template parameter removed?

This was a hint to the implementation to prioritize for lowered memory usage or performance specifically. In other implementations this could've, for example, been used to switch between a jump-counting skipfield and the bitfield+jump-counting approach described in the [alternative implementations appendix](#) (reducing memory cost at cost of performance). In the early reference implementation, this told the container to switch between 16-bit and 8-bit skipfield types (smaller bitdepth skipfield types limit the block capacities due to the constraints of the jump-counting skipfield pattern). However, prior to a particular LEWG thread there had not been sufficient benchmarking and memory testing done on this.

When more thorough benchmarking including memory assessments were done, it was found that the vast bulk of unnecessary memory usage came from erased elements in hive when an active block was not yet empty (and therefore freed to the OS or reserved, depending on implementation), rather than the skipfield type. This meant that making block capacities appropriately-sized was more important to performance and cache friendliness than the skipfield type, as - assuming a randomised erasure pattern - smaller blocks were more likely to become empty and therefore be removed from the iterative sequence than larger blocks, thereby improving element contiguity and iteration performance by skipblocks. But also, reusing erased element memory space in existing element blocks was much faster than having to deallocate/reserve blocks and subsequently allocate or un-reserve new element blocks, so, having too-small element block capacities was also bad.

The only place where the `sizeof(skipfield_type)` turned out to be more important than block capacities was when using small types such as scalars, where the additional memory use from the skipfield (proportional to the type size) was significant, and so reducing the skipfield_type from 16-bits to 8-bits improved performance even though it made the max block capacities very small (255 elements), due to cache effects.

As a result of all this it was decided to remove the priority parameter and let implementations decide when to switch internal mechanisms, rather than relying on user guesswork; probably a better approach. I think the priority parameter might've been useful for additional compile-time decision processes, such as deciding what type of block retention strategy to use when an active block becomes empty of elements. Also, having a priority tag gave the ability to specify new priority values in future as part of the standard, potentially allowing for big changes without breaking ABI. However, it is also a matter of complexity vs simplicity, and simplicity is worthwhile too.

17. Why does a bidirectional container have iterator operators `>`, `<`, `>=`, `<=` and `<=>`?

These are useful for several reasons:

- a. Where the user is iterating over the container elements and adding/subtracting a greater-than-1 value to/from the iterator for each cycle of the loop, having `!= end()` or `!= another_iterator` would not necessarily work as a loop end condition.
- b. It can be used by the user to correctly order two iterators obtained from random parts of the sequence, before supplying them to a function covering a range eg. `range-insertion`, `std::distance`, `range-erasure`, and so on. This stops the function throwing an exception when `it1 > it2`.
- c. Because hive insertion/emplacement location is unspecified, if you have a specific range of elements you're interested in (perhaps you've calculated the distance between two iterators and that distance value is important somehow), iterator ops `<`, `<=`, `>`, `>=` are the only way to determine whether an element you just inserted is now within the range. Likewise if external objects/entities are removing elements from a hive via stored pointers/iterators, the comparison operations above are the only way to determine if the element just erased was within the given range.

18. Why is the insertion time complexity for singular `insert/emplace` $O(1)$ amortized as opposed to $O(1)$?

In the current reference implementation (at time of writing) in the event of needing to allocate a new active block or change a reserved block into an active block, there is an update of active block numbers which may occur if the current back block has a group number == `std::numeric_limits<size_type>::max()`. The occurrence of this event is 1 in every `std::numeric_limits<size_type>::max()` block allocations/transfers-from-reserved (ie. basically *never* in the lifetimes of almost all programs). The number of active blocks at this point could be small or large depending on how many blocks have been deallocated in the past.

If a hive were implemented as a vector of pointers to blocks instead, this would also necessitate amortized time complexity as when the vector became full and more blocks were needed, all element block pointers would need to be reallocated to a new memory block.

19. Why does throwing an exception when calling `reshape` allow for the resultant `current - limits` to be something other than the original `current - limits` and the user-supplied `block_limits`?

If reshaping a hive instance from, for example, a `current - limits` of min = 6 elements, max = 12 elements, to a supplied user limit of min = 15 elements, max = 255 elements, this basically means none of the original blocks fit within the new limits. So all elements have to go into new blocks. A very basic strategy would be to allocate all new active blocks before copying elements, then deallocating all old active blocks once elements have been copied over. However if there are many active blocks, this could potentially lead to out-of-memory exceptions. So an implementation might choose to allocate blocks one-at-a-time, copy elements into it, then deallocate each old block as it becomes empty.

If the latter strategy is pursued we don't necessarily have the old memory blocks to revert to if an exception is thrown and we're halfway through copying the elements over. And we wouldn't want to start creating new element blocks of the old capacity and copy the elements back to them, as this risks yet more exceptions! The best strategy in this scenario is to keep all existing blocks, remove duplicate elements if any exist, and change `current - limits` to values which accomodate both the original block capacities and the new block capacities.

20. Why does the `sort()` specification require that `T` is `Cpp17MoveInsertable` and `Cpp17MoveAssignable` into `*this`?

`Cpp17MoveAssignable` is obvious, `Cpp17MoveInsertable` is to deal with the event that a specific sorting technique wants to take advantage of erased element memory spaces between active elements. For example, for the hive sequence: 4, 3, 5 - where there is an erased element memory space before 4 - the sort technique could simply move-construct the 3 to the erased element memory space instead of swapping with the 4, saving instructions and the creation of a temporary. Also, the sort routine could choose to use erased element memory spaces, if they exist, or empty space at the back of the container, as space to store temporary buffers for swap operations instead of dynamically-allocating buffers - this may be useful both to performance and memory use, if elements are very large.

21. Reasoning for container name?

See paper P2332R0.

22. **"Unordered and no associative lookup, so this only supports use cases where you're going to do something to every element."**

The container was originally designed for highly object-oriented situations where you have many elements in different containers linking to many other elements in other containers. This linking can be done with pointers or iterators in hive (insert returns an iterator which can be dereferenced to get a pointer, pointers can be converted into iterators with `get_iterator` (for use with `erase`)) and because pointers/iterators stay stable regardless of insertion/erasure to other elements, this usage is unproblematic. You could say the pointer is equivalent to a key in this case but without the overhead. That is the first access pattern, the second is straight iteration over the container. However, the container can have (typically better than $O(n)$) `std::advance/next/prev` overloads, so multiple elements can be skipped efficiently.

23. **"Prove this is not an allocator"**

I'm not really sure how to answer this, as I don't see the resemblance, unless you count maps, vectors etc as being allocators also. The only aspect of it which resembles what an allocator might do, is the memory re-use mechanism. It would be impossible for an allocator to perform a similar function while still allowing the container to iterate over the data linearly in memory, preserving locality, in the manner described in this document.

24. **"If this is for games, won't game devs just write their own versions for specific types in order to get a 1% speed increase anyway?"**

This is true for many/most AAA game companies who are on the bleeding edge, as well as some of the more hardcore indie developers, but they also do this for vector etc, so they aren't the target audience of `std::` for the most part; sub-AAA game companies are more likely to use `std::`/third party/pre-existing tools. Also, this type of structure crops up in [many fields](#), not just game dev. So the target audience is probably everyone other than people working on bare metal core loops in extreme-high-performance circumstances, but even then, it facilitates communication across fields and companies as to what this type of container is, giving it a standardized name and understanding.

25. **"Is there active research in this problem space? Is it likely to change in future?"**

The only analysis done has been around the question of whether it's possible for this specification to fail to allow for a better implementation in future. Bucket arrays have been around since the 1990s at least, there's been no significant innovation in them until now. I've been researching/working on hive since early 2015, and while I can't say that a better implementation might not be possible, I am confident that no change should be necessary to the specification to allow for future implementations. If a change were necessary it would likely be a loosening of the specification rather than a breaking change. This is because of the [C++ container requirements and how these constrain implementation](#).

The requirement of allowing no reallocations upon insertion or erasure, truncates possible implementation strategies significantly. Element blocks have to be independently allocated so that they can be removed from the iterative sequence (when empty) without triggering reallocation of subsequent elements. There's limited numbers of ways to do that and keep track of the element blocks at the same time. Erased element locations must be recorded (for future re-use by insertion) in a way that doesn't create allocations upon erasure, and there's limited

numbers of ways to do this also. Multiple consecutive erased elements have to be skipped in $O(1)$ time in order for the iterator to meet the C++ iterator $O(1)$ function requirement, and again there're limits to how many ways you can do that. That covers the three core aspects upon which this specification is based. See the [alt implementation appendix](#) for more information.

26. "We have (full container) splice, unique and sort, like in `std::list`, but not merge?"

With splice and unique we can retain the guarantee that pointers to non-erased elements stay valid (sort does not guarantee this for hive), but with merge we cannot, as the function requires an interleaving of elements, which is impossible to accomplish without invalidating pointers, unless the elements are allocated individually. This is not the case in hive, hence including merge may confuse users as to why it doesn't share that same property of valid pointers with `std::list`. `std::sort` however is known to invalidate pointers when used with vectors and deques, so `sort()` as a member function does not necessarily have the association of retaining pointer validity.

27. Why no `resize()`?

This was a choice by LEWG to avoid confusing the user, as insertion position into a hive is implementation-defined. In the case of hive, resizing would not necessarily insert new elements to the back of the container, when the supplied size was larger than the existing `size()`. New elements could be inserted into erased elements memory locations. This means the initialization of those non-contiguous elements (if they are POD types) cannot be optimized by use of `memset`, removing the main performance reason to include `resize()`. The lack of ability to specify the position of insertion removes the "ease of developer use" reason to include `resize()`.

28. "Why not support `push_back` and `push_front`?"

- a. Ordered insertion would create performance penalties due to not reusing previously-erased element locations, which in turn increases the number of block allocations necessary and reduces iteration speed due to wider gaps between active elements and the resultant reduced cache locality. This negates the performance benefits of using this container.
- b. Newcomers will get confused and use `push_back` instead of `insert`, because they will assume this is faster based on their experience of other containers, and the function call itself may actually be faster in some circumstances. But it will also inhibit performance for the reasons above. Further, explaining how the container works and operates has proved to be difficult even with C++ committee members, so being able to explain it adequately to novices such that they avoid this pitfall would not be guaranteed.
- c. It should be unambiguous as to its interface and how it works, and what guarantees are in place. Making insertion guarantees straightforward is key to performant usage. Having fewer constraints is also important for allowing future, potentially-faster, implementations.
- d. There are better containers for ordered insertion eg. `deque`/`vector`/`plf::list`.

29. "Why not `constexpr`?" (yet)

At time of writing constexpr containers are still a new-ish thing and some of the kinks of usage may yet have to be worked out. Early compiler support was not good but this has improved steadily over time. I wasn't happy with having to label each and every function as constexpr, but there seem to be movements toward labeling classes as a whole as constexpr, so if that comes through it will remove that problem. Having said that, there are a couple of things to consider when designing a constexpr hive. One is that in the reference implementation there is extensive use of reinterpret_cast'ing pointers, mainly for two areas:

1. The per-block free-list of erased elements for later use by insert/assign/emplace/etc.
2. Allocating the element block and associated skipfield block in a single allocation, increasing performance.

As reinterpret_cast is not allowed at compile time, 1 could be worked around by creating a union between the element type and the free list struct/pair. 2 would not be possible at compile time, and the element block and skipfield blocks would have to be allocated separately. So it is *possible*, though it could be hard work, and may decrease runtime performance.

Constexpr containers do not allow non-literal member variables, and in the reference implementation the begin() and end() iterators are stored as member variables and also function as records of the first and last active blocks in the sequence. This much can be worked around by breaking the iterators apart into their individual pointers, then reconstructing them on the fly whenever begin()/end()/etc are called.

For the moment I am happier for std::array and std::vector to be the "canaries in the coalmine" here.

30. **Licensing for the reference implementation (zLib) - is this compatible with libstdc++/libc++/MS-STL usage?**

Yes. [zLib](#) license is compatible with both [GPL3](#) and [Apache](#) licenses (libc++/MS-STL). zLib is a more permissive license than all of these, only requiring the following:

This software is provided 'as-is', without any express or implied warranty. In no event will the authors be held liable for any damages arising from the use of this software.

Permission is granted to anyone to use this software for any purpose, including commercial applications, and to alter it and redistribute it freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not claim that you wrote the original software. If you use this software in a product, an acknowledgment in the product documentation would be appreciated but is not required.
2. Altered source versions must be plainly marked as such, and must not be misrepresented as being the original software.
3. This notice may not be removed or altered from any source distribution.

Please note that "product" in this instance doesn't mean 'source code', as in a library, but a program or executable. This is made clear by line 3 which clearly differentiates source distributions from products.

Representatives from libc++, libstdc++ and MS-STL have stated they will likely either use the reference or use it as a starting point, and that licensing is unproblematic (with the exception of libc++ who stated they would need to run it past LLVM legal reps). However if in any case licensing becomes problematic as the sole author of the reference implementation [I am in a position to grant use of the code](#) under other licenses as needed.

31. How does hive solve the ABA problem where a given iterator/pointer points to a given element, then that element is erased, another element is inserted and re-uses the previous element's memory location? We now have invalidated iterators/pointers which point to valid elements.

It doesn't. Detecting these cases is the end user's responsibility, as it is in deque or vector when elements are erased. In the case of hive I would recommend the use of either a unique ID within the element itself. The end user can then build their own "handle" wrapper around a pointer or iterator which stores a copy of this ID, then compares it against the element itself upon accessing it to see if it's the same.

In terms of guarantees that an element has not been replaced via hive usage, replacement may occur if:

- a. Any number of erasures have occurred, and then at least one insertion has occurred.
- b. clear() has been called and then at least one insertion has occurred.
- c. shrink_to_fit(), reshape(), assign(), sort() or iterator operator = have been called.

32. Asides from it already being a known container type in many domains, and it's performance vs other standard library containers, what are good reasons to standardise hive?

- a. "Build it and they will come" - quote from the movie "Field of Dreams" which basically means, people don't always know they want something until it's there. Once this is available and people understand the advantages (performance, memory, pointer validity etc) they are likely to use it. Particularly in fields where people are doing long-term compatibility/cross-platform work they are unlikely to use non-std:: containers, even if there are significant advantages, however if it's in the standard, they may.
- b. std::list. Developers commonly use this for situations where they need stable pointers to elements regardless of insertion/erasure ([here](#) is an example in libreoffice, line 125 - std::list is used extensively in 50 libreoffice source files), but it is cache-unfriendly, slow for the majority of scenarios and wasteful in terms of memory. Hive is cache-friendlier, faster and (in terms of reference implementation) uses only one skipfield_type (8/16-bit) per element to maintain it's functionality, as opposed to 2-pointers per-element for std::list. The main advantage of std::list is it's ordered non-back/front insertion.
- c. Other languages may beat us to it. I know that one person is currently developing a hive-equivalent for Rust.
- d. Having this as std:: will allow greater communication and consistency across domains and organisations.

33. What are the advantages of the user being able to reserve() when an allocator can mitigate some of the effects of allocating smaller blocks rather than larger ones?

An allocator can only decrease the number of allocation calls to the OS. While it might allocate one small block contiguous to another in the order of the sequence, it also might not (and likely won't), which decreases iteration speed. Further, there is a certain amount of metadata necessary for each block (regardless of implementation), which needs to be updated etc when erasures/insertions happen. Hence, by having more blocks than you need, you

also increase memory overhead. There is also procedural overhead associated with each block, in terms of many of the operations like splice, reshape and reserve, where the more blocks you have, the more operations you incur (though the cost is typically very low).

34. **This seems over-specified.**

Places to start: read the first paragraph of the introduction to this paper, then the Design Decisions section, then the [constraints](#) and [alt implementation](#) appendices.

Most of the specificity comes from the type of container and the C++ standard's specifications. In terms of this type of container, this paper represents to the best of my knowledge the widest scope of implementation while still fulfilling the core invariants of the container, maintaining reasonable performance, and satisfying the C++ standard requirements.

There is also a risk of underspecification. The (at time of writing) MS STL version of deque allocates blocks in a fixed size of 16 bytes so any element type above 8 bytes makes it a linked list - which it is allowed to do by the standard, which does not allow the user to specify block capacities. There are advantages to being more specific when you get to something more complex than an array because it encourages good implementation practice. As a result we attempt to reach a balance between under/over-specification.

35. **What is the reasoning behind which operations do/don't copy current - limits between hives?**

These generally follow the pattern for allocators - which makes sense as their use may have a relationship with user-supplied allocator constraints. They're transferred during move and copy construction, operator = && and swap, but not during splice or operator = &. Unlike allocators, they are not able to be specified in the copy/move constructors, which makes sense for the move constructor since it would have to throw if the transferred blocks did not fit within the specified limits. If the user wants to specify new capacity limits when copying from another hive, they can do the following instead of calling a copy constructor: `hive<int> h(hive_limits(10, 50));`

```
h = other_hive;
```

Likewise if the user wants to specify new capacity limits when moving from another hive, they could:

```
hive<int> h(std::move(other_hive));
```

```
h.reshape(10, 50);
```

Appendix D - Typical game engine requirements

Here are some more specific requirements with regards to game engines, verified by game developers within SG14:

- a. Elements within data collections link to elements within other data collections (through a variety of methods). These links must stay valid throughout the course of the game/level. Any container where most of its operations cause pointer/index invalidation tend to create difficulties and necessitates workarounds.
- b. The majority of data is simply iterated over, transformed, linked to and utilized with no regard to order.
- c. Erasing or otherwise "deactivating" objects occurs frequently in performance-critical code. For this reason methods of erasure or deactivation which create strong performance penalties are avoided.

- d. Inserting new objects in performance-critical code (during gameplay) is common - for example, a tree drops leaves, or a player spawns in an multiplayer game.
- e. It is not always clear in advance how many elements there will be in a container at the beginning of development, or at the beginning of a level during play. Genericized game engines in particular have to adapt to considerably different user requirements and scopes. For this reason extensible containers which can expand and contract in realtime are useful.
- f. Due to the effects of cache on performance, memory storage which is more-or-less contiguous is preferred.
- g. Memory waste is avoided.

`std::vector` in its default state does not meet these requirements due to:

- 1. Poor single insertion performance (regardless of insertion position) due to the need for reallocation upon reaching capacity.
- 2. Insert invalidates pointers/iterators to all elements
- 3. Erase invalidates pointers/iterators/indexes to all elements after the erased element

Game developers therefore tend to either develop custom solutions for each scenario or implement workarounds for vector. The most common workarounds are most likely the following or derivatives thereof:

- 1. Using a boolean flag or similar to indicate the inactivity of an object (as opposed to actually erasing from the vector). Elements flagged as inactive are skipped during iteration. A free list may be used in some situations.

Advantages: Fast "deactivation". Easy to manage in multi-threaded environments. Memory cost can be low if a bitfield is used.

Disadvantages: Can be slower to iterate due to branching code, potential bit access, and unknown number of inactive elements between any two active elements. The latter creates variable latency when iterating and violates time complexity requirements for C++ iterator operations `++/--`.

- 2. Using a deque of elements and a vector of indexes. Since deques don't reallocate during insertion, this preserves stable pointers to elements post-insertion. When erasing, the erasure occurs only in the vector of indexes, not the deque of elements, which retains a 'hole' in the deque where the destructed element was. It may use some form of free list in the deque in order to re-use erased element memory during later insertions. When iterating, one iterates over the vector and accesses the elements from the deque via the indexes. Erasure involves destructing the element in the deque, adding that element space to the free list, then swapping the element's index in the vector with the back index, and popping the new back index (the latter avoids slowdown and jitter due to reallocation). As an alternative one could store a vector-of-elements instead of a deque, but then the external objects storing links to the elements must instead store an index into the vector-of-elements (since pointers would be invalidated upon insertion when `capacity() == size()`) and be able to access the vector directly.

Advantages: Fast iteration.

Disadvantages: Insertion to the vector may reallocate indexes which is slow - can be replaced with a deque to avoid this, at a cost to iteration. Indexes must be at system bit-depth to accomodate all possible capacities, therefore the memory waste can be large - particularly for small types.

- 3. Combining a swap-with-back-element-and-pop approach to erasure with some form of dereferenced lookup system to enable contiguous element allocation (sometimes called a [Packed array](#)).

Advantages: Iteration is at standard vector speed.

Disadvantages: Erasure will be slow if objects are large and/or non-trivially copyable, thereby making swap costs large. All link-based access to elements incur additional costs due to the dereferencing system. Stable references to elements cannot be maintained via iterators nor pointers, but must be made using a third lookup object which refers to the lookup table - since the lookup table itself doesn't refer to elements in contiguous order and is thereby

unsuitable for iteration. Further, the lookup table, regardless of whether it uses pointers or indexes, needs to be at system bit-depth, which wastes memory - particularly for small types.

Hive brings a more generic solution to these contexts with better performance according to my benchmarks.

Appendix E - User experience reports

Richard, Creative Assembly:

"I'm the lead of the Editors team at Creative Assembly, where we make tools for the Total War series of games. The last game we released was Three Kingdoms, currently doing quite well on Steam. The main tool that I work on is the map creation editor, kind of our equivalent of Unreal Editor, so it's a big tool in terms of code size and complexity.

The way we are storing and rendering entities in the tool currently is very inefficient: essentially we have a quadtree which stores pointers to the entities, we query that quadtree to get a list of pointers to entities that are in the frustum, then we iterate through that list calling a virtual draw() function on each entity. Each part of that process is very cache-unfriendly: the quadtree itself is a cache-unfriendly structure, with nodes allocated on the heap, and the entities themselves are all over the place in memory, with a virtual function call on top.

So, I have made a new container class in which to store the renderable versions of the entities, and this class has a bunch of colonies inside, one for each type of 'renderable'. On top of this, instead of a quadtree, I now have a virtual quadtree. So each renderable contains the index of the quadtree node that it lives inside. Then, instead of asking the quadtree what entities are in the frustum, I ask the virtual quadtree for a node mask of the nodes what are in the frustum, which is just a bit mask. So when rendering, I iterate through all the renderables and just test the relevant bit of the node mask to see if the renderable is in the frustum. (Or more accurately, to see if the renderable has the potential to be in the frustum.) Nice and cache friendly.

When one adds an entity to the container, it returns a handle, which is just a pointer to the object inside one of the colonies returned as a `std::uintptr_t`. So I need this to remain valid until the object is removed, which is the other reason to use a colony."

Andrew Shuvalov, MongoDB:

"I implemented a standalone open source project for the thread liveness monitor: <https://github.com/shuvalov-mdb/thread-liveness-monitor>. Also, I've made a video demo of the project: <https://youtu.be/uz3uENpjRfA>

The benchmarks are in the doc, and as expected the `plf::colony` was extremely fast. I do not think it's possible to replace it with any standard container without significant performance loss. Hopefully, this version will be very close to what we will put into the MongoDB codebase when this project is scheduled."

Daniel Elliot, Weta Digital:

"I'm using it as backing storage for a volumetric data structure (like openvdb). Its sparse so each tile is a 512^3 array of float voxels.

I thought that having colony will allow me to merge multiple grids together more efficiently as we can just splice the tiles and not copy or reallocate where the tiles don't overlap. Also adding and removing tiles will be fast. Its kind of like using an arena allocator or memory pool without having to actually write one."

Note: this is a private project Daniel is working on, not one for Weta Digital.

Gašper Ažman, Citadel Securities:

"Internally we use it as a slab allocator for objects with very different lifetime durations where we want aggressive hot memory reuse. It lets us ensure the algorithms are correct after the fact by being able to iterate over the container and verify what's alive.

It's a great single-type memory pool, basically, and it allows iteration for debugging purposes :)

Where it falls slightly short of expectation is having to iterate/delete/insert under a lock for multithreaded operation - for those usecases we had to do something different and lock-free, but for single-threaded applications it's amazing."

Appendix F - A brief and incomplete guide for selecting the appropriate container from inside/outside the C++ standard library, based on performance characteristics, functionality and benchmark results

Guides and flowcharts I've seen online have either been performance-agnostic or incorrect. This is not a perfect guide, nor is it designed to suit all participants, but it should be largely correct in terms of its focus. Note, this guide does not cover:

- a. All known C++ containers
- b. Multithreaded usage/access patterns in any depth
- c. All scenarios
- d. The vast variety of map variants and their use-cases
- e. Examinations of technical nuance (eg. at which sizeof threshold on a given processor does a type qualify as large enough to consider not using it in a vector if there is non-back erasure?). For that reason I'm qualifying 'Very large' or 'large' descriptors in this guide.

These are broad strokes and should be treated as such. Specific situations with specific processors and specific access patterns may yield different results. There may be bugs or missing information. The strong insistence on arrays/vectors where possible is for code simplicity, ease of debugging, and performance via cache locality. For the sake of brevity I am purposefully avoiding any discussion of the virtues/problems of C-style arrays vs `std::array` or `vector` here. The relevance of all assumptions are subject to architecture. The benchmarks this guide is based upon are available [here](#), [here](#). Some of the map/set data is based on [google's abseil library documentation](#).

Start!

a = yes, b = no

0. Is the number of elements you're dealing with a fixed amount?
 - 0a. If so, is all you're doing either pointing to and/or iterating over elements?
 - 0aa. If so, use an array (either static or dynamically-allocated).
 - 0ab. If not, can you change your data layout or processing strategy so that pointing to and/or iterating over elements would be all you're doing?
 - 0aba. If so, do that and goto 0aa.
 - 0abb. If not, goto 1.
 - 0b. If not, is all you're doing inserting-to/erasing-from the back of the container and pointing to elements and/or iterating?
 - 0ba. If so, do you know the largest possible maximum capacity you will ever have for this container, and is the lowest possible maximum capacity not too far away from that?
 - 0baa. If so, use vector and reserve() the highest possible maximum capacity. Or use boost::static_vector for small amounts which can be initialized on the stack.
 - 0bab. If not, use a vector and reserve() either the lowest possible, or most common, maximum capacity. Or boost::static_vector.
 - 0bb. If not, can you change your data layout or processing strategy so that back insertion/erasure and pointing to elements and/or iterating would be all you're doing?
 - 0bba. If so, do that and goto 0ba.
 - 0bbb. If not, goto 1.
 1. Is the use of the container stack-like, queue-like or ring-like?
 - 1a. If stack-like, use plf::stack, if queue-like, use plf::queue (both are faster and configurable in terms of memory block sizes). If ring-like, use [ring_span](#) or [ring_span lite](#).
 - 1b. If not, goto 2.
 2. Does each element need to be accessible via an identifier ie. key? ie. is the data associative.
 - 2a. If so, is the number of elements small and the type sizeof not large?
 - 2aa. If so, is the value of an element also the key?
 - 2aaa. If so, just make an array or vector of elements, and sequentially-scan to lookup elements. Benchmark vs absl:: sets below.
 - 2aab. If not, make a vector or array of key/element structs, and sequentially-scan to lookup elements based on the key. Benchmark vs absl:: maps below.
 - 2ab. If not, do the elements need to have an order?
 - 2aba. If so, is the value of the element also the key?
 - 2abaa. If so, can multiple keys have the same value?
 - 2abaaa. If so, use absl::btree_multiset.
 - 2abaab. If not, use absl::btree_set.
 - 2abab. If not, can multiple keys have the same value?
 - 2ababa. If so, use absl::btree_multimap.
 - 2ababb. If not, use absl::btree_map.
 - 2abb. If no order needed, is the value of the element also the key?
 - 2abba. If so, can multiple keys have the same value?
 - 2abbba. If so, use std::unordered_multiset or absl::btree_multiset.
 - 2abbab. If not, is pointer stability to elements necessary?
 - 2abbaba. If so, use absl::node_hash_set.
 - 2abbabb. If not, use absl::flat_hash_set.
 - 2abbbb. If not, can multiple keys have the same value?
 - 2abbbaa. If so, use std::unordered_multimap or absl::btree_multimap.
 - 2abbbbb. If not, is on-the-fly insertion and erasure common in your use case, as opposed to mostly lookups?
 - 2abbbbba. If so, use [robin-map](#).
 - 2abbbbbb. If not, is pointer stability to elements necessary?

2abbbba. If so, use `absl::flat_hash_map<Key, std::unique_ptr<Value>>`. Use `absl::node_hash_map` if pointer stability to keys is also necessary.

2abbbbbb. If not, use `absl::flat_hash_map`.

2b. If not, goto 3.

Note: if iteration over the associative container is frequent rather than rare, try the `std::` equivalents to the `absl::` containers or [tsl::sparse_map](#). Also take a look at [this page of benchmark conclusions](#) for more definitive comparisons across more use-cases and hash map implementations.

3. Are stable pointers/iterators/references to elements which remain valid after non-back insertion/erasure required, and/or is there a need to sort non-movable/copyable elements?

3a. If so, is the order of elements important and/or is there a need to sort non-movable/copyable elements?

3aa. If so, will this container often be accessed and modified by multiple threads simultaneously?

3aaa. If so, use `forward_list` (for its lowered side-effects when erasing and inserting).

3aab. If not, do you require range-based splicing between two or more containers (as opposed to splicing of entire containers, or splicing elements to different locations within the same container)?

3aaba. If so, use `std::list`.

3aabb. If not, use `plf::list`.

3ab. If not, use `hive`.

3b. If not, goto 4.

4. Is the order of elements important?

4a. If so, are you almost entirely inserting/erasing to/from the back of the container?

4aa. If so, use `vector`, with `reserve()` if the maximum capacity is known in advance.

4ab. If not, are you mostly inserting/erasing to/from the front of the container?

4aba. If so, use `deque`.

4abb. If not, is insertion/erasure to/from the middle of the container frequent when compared to iteration or back erasure/insertion?

4abba. If so, is it mostly erasures rather than insertions, and can the processing of multiple erasures be delayed until a later point in processing, eg. the end of a frame in a video game?

4abbaa. If so, try the `vector erase_if` pairing approach listed at the bottom of this guide, and benchmark against `plf::list` to see which one performs best. Use `deque` with the `erase_if` pairing if the number of elements is very large.

4abbab. If not, goto 3aa.

4abbbb. If not, are elements large or is there a very large number of elements?

4abbbba. If so, benchmark `vector` against `plf::list`, or if there is a very large number of elements benchmark `deque` against `plf::list`.

4abbbb. If not, do you often need to insert/erase to/from the front of the container?

4abbbba. If so, use `deque`.

4abbbbb. If not, use `vector`.

4b. If not, goto 5.

5. Is non-back erasure frequent compared to iteration?

5a. If so, is the non-back erasure always at the front of the container?

5aa. If so, use `deque`.

5ab. If not, is the type large, non-trivially copyable/movable or non-copyable/movable?

5aba. If so, use `hive`.

5abb. If not, is the number of elements very large?

5abba. If so, use a `deque` with a swap-and-pop approach (to save memory vs `vector` - assumes standard `deque` implementation of fixed block sizes) ie. when erasing, swap the element you wish to erase with the back element, then `pop_back()`. Benchmark vs `hive`.

5abbbb. If not, use a `vector` with a swap-and-pop approach and benchmark vs `hive`.

5b. If not, goto 6.

6. Can non-back erasures be delayed until a later point in processing eg. the end of a video game frame?

6a. If so, is the type large or is the number of elements large?

6aa. If so, use hive.

6ab. If not, is consistent latency more important than lower average latency?

6aba. If so, use hive.

6abb. If not, try the `erase_if` pairing approach listed below with vector, or with deque if the number of elements is large. Benchmark this approach against hive to see which performs best.

6b. If not, use hive.

Vector erase_if pairing approach:

Try pairing the type with a boolean, in a vector, then marking this boolean for erasure during processing, and then use `erase_if` with the boolean to remove multiple elements at once at the designated later point in processing. Alternatively if there is a condition in the element itself which identifies it as needing to be erased, try using this directly with `erase_if` and skip the boolean pairing. If the maximum number of elements is known in advance, use vector with `reserve()`.

Appendix G - Hive constraints summary.

This is a summary of information already contained within P0447.

Constraints forced by the C++ Standard:

1. Iterator operations must be $O(1)$ amortized (`iterator.requirements.general`), forces:

- Non-boolean skipping mechanism (unlike real-world implementations of bucket-lists etc) eg. jump-counting skipfield (only known possibility at present point in time) or better. For a boolean skipfield there is an undefined number of branching statements (ie. erased elements to check for and skip over) for each `++/--` operation, worst case linear in capacity. See Design Decisions, 'A method of skipping multiple erased elements in $O(1)$ time during iteration'.
 - this, in turn enables faster, non-branching iteration with more consistent latency, and faster range-erasure/range-insertion.
- Removal of element blocks from iterative sequence once they become empty of erased elements, otherwise skipping over an undefined number of empty blocks may occur during `++/--` operations, making those operations (in worst case) linear in the total number of blocks in terms of time complexity.
 - this, in turn, forces storage of block metadata detailing either (a) number of non-erased elements in block or (b) number of erased elements in block, to be matched with block capacity to determine block emptiness. Knowledge of each block's capacity is also necessary and may need to be recorded for each block.

2. No exceptions allowed upon erase (`container.rev.reqmts`), forces:

- Free-list of erased elements or free-list of skipblocks as the erased-element-location-recording mechanism, as opposed to a stack/vector/etc of pointers to erased element locations or similar, as the latter creates occasional allocations upon erase.

Constraints forced by container type (most real-world implementations of bucket-lists etc):

1. Stable element memory locations (the reasons for which are described in the Introduction and Motivation sections of this paper), forces:
 - no reallocation upon insert or erase, and for most other operations.
 - only linked-list/tree of individually-allocated elements, linked list of element blocks + skipping structure, or container of pointers to element blocks + skipping structure (only known possibilities) meet this requirement without strong memory use - see Design Decisions section '1. Collection of element blocks + metadata', and the [game engine requirements appendix](#) for more info.
2. High-speed/predictable-latency iteration, forces:
 - element blocks + skipping structure as opposed to linked-list/tree of individually-allocated elements, due to better memory locality.
 - removal of active blocks from iterative sequence once they become empty of erased elements, otherwise there would be skips over potentially many empty blocks during a single ++ operation.
3. High-speed/predictable-latency insert, forces:
 - element blocks + skipping structure as opposed to linked-list/tree of elements due to lower number of allocations.
 - per-element-block skipping structures rather than global ones. A global one will have vector characteristics and force $O(n)$ reallocation when insert triggers the creation of a new element block.
4. High-speed/predictable-latency erase, forces:
 - element blocks + skipfields as opposed to linked-list/tree of elements due to lower number of deallocations.
 - per-element-block skipping structures rather than a global one. A global one will have vector characteristics and force $O(n)$ reallocation when removing an element block from the iterative sequence, when that element block becomes empty of non-erased elements.
 - per-element-block free lists of erased elements as opposed to a global free list. A global one would mean that, upon removal of a block from the iterative sequence (when it becomes empty of non-erased elements), hive would have to traverse the entirety of the global free list ($O(n)$ in the number of erased elements within the hive) in order to remove all the free list entries from that block.

Constraints forced by colony/hive (compared to some bucket array-type implementations):

1. Higher-speed iteration/insert and less memory usage, forces:
 - Reuse of erased element locations rather than simply erasing till an element block becomes empty, then removing the block (not re-using locations lowers element locality, increases number of block deallocations/allocations and memory usage). Some bucket-array-like structures also do this. See Design Decisions, 'Erased-element location recording mechanism'.
 - this, in turn, forces random positions for insertions. Bucket-array structures without re-use can guarantee insertion at back of container, at the cost of iteration speed and memory.
 - Growth factor for element block capacities (up to limit) unless user specifies min/max block capacity limits as equal. This reduces number of allocations and increases element locality compared to static block capacities, when user does not know their max amount of elements in advance of construction.

- this, in turn, forces storage of metadata on capacity for each element block.
- It also provides the facility for a user to specify their own block limits with minimal change to an implementation.

2. >, <, >=, <= and <=> iterator operators, primarily to allow for ease-of-use with loops with greater-than-1 iterator increments/decrements, forces:

- Depending on implementation, storage of element block number (ordering of active block in iterative sequence) metadata may be necessary. In reference implementation this number only needs to be updated once in every `std::numeric_limits<size_type>::max()` element block allocations, and occasionally during splice. Other styles of implementation might not need these.

So in order to serve the requirements of high performance, stable memory locations, and the C++ standard, a standard library implementation of this type of container is quite constrained as to how it can be specified. Ways of meeting those constraints which deviate from reference implementation are detailed in the [alt implementations appendix](#).

Appendix H - Prior art links

In addition to the stuff below I have written a supporting paper which attempts to assess prevalence of this type of container within the programming industry - with the results being about 61% of respondents using something like it (see [P3011](#)).

Sean Middleditch talks about 'arrays with holes' on his old blog, which is similar but using id lookup instead of pointers. There is some code: [link](#)

Jonathan Blow talks about bucket arrays here (note, he says "I call it a bucket array" which might connote that it's his concept depending on your interpretation, but it's just an indication that there are many names for this sort of thing): [link](#)

A github example (no iteration, lookup-only based on entity id): [link](#)

This guy describes free-listing and holes in 'deletion strategies': [link](#)

Similar concept, static array with 64-bit integer as bit-field for skipping: [link](#)

Going over old colony emails I found someone whose company had implemented something like the above but with atomic 64-bit integers for boolean (bitfield) skipfields and multi-blocks for multithreaded use.

I initially thought while I was developing this that it was a newish concept, but particularly after doing the cppcon talk, more people kept coming forward saying, yes, we do this, but with *X* specific difference.

Pool allocators etc are often constructed similarly to hives, at least in terms of using free lists and multi memory blocks. However they are not useful if you have large amounts of elements which require bulk processing for repeated timeframes, because an allocator doesn't provide iteration, and manually iterating via say, a container of pointers to objects in a pool has the same performance and memory-use issues as linked lists.

Appendix I - Information on non-reference-implementation hive designs

Core aspect design alternatives

See the [Design Decisions](#) section for a revision on these and how the reference implementation applies them, and below for the alternatives.

1. Collection of element blocks + metadata

It is possible to implement a hive via a vector of pointers to blocks+metadata. Some of the metadata could be stored with the pointers in the vector. More analysis of this approach is described in the section below this one, [A full implementation guide using the vector-of-pointers-to-blocks approach](#).

2. A method of skipping erased elements in $O(1)$ time during iteration

The low-complexity jump-counting pattern used in the reference implementation has a lot of similarities to the [high complexity jump-counting pattern](#), which was a pattern previously used by the [original reference implementation](#). Using the high-complexity pattern is an alternative, though the skipfield update time complexity guarantees for insertion/erasure with that pattern are at-worst $O(n)$ in the number of elements in the block. In practice the majority of those updates constitute a single memcopy operation which may resolve to a much smaller number of operations at the hardware level. But it is still slightly slower than the low-complexity jump-counting pattern (around 1-2% in my benchmarking).

A pure boolean skipfield is not usable because it makes iteration time complexity at-worst $O(n - 2)$ in the summed capacity of two blocks (ie. a non-erased element at the beginning of one block, with no subsequent non-erased elements till the end of the next block). This can result in thousands of branching statements & skipfield reads for a single ++ operation in the case of many consecutive erased elements. In the high-performance fields for which this container was initially designed, this brings with it unacceptably unpredictable latency.

However another strategy combining a low-complexity jump-counting pattern *and* a boolean skipfield, which saves memory at the expense of computational efficiency, is possible while preserving $O(1)$ iterator operations. There is a simpler version of this, and a more complex version - both of which have some advantages.

Bitfield + jump-counting pattern - simple variant:

- Instead of storing the data for the jump-counting pattern in its own skipfield, have a boolean bitfield indicating which elements are erased - 0 for non-erased, 1 for erased. Store the jump-counting data in the erased element's memory space (possibly alongside free list data).
- When iterating, check whether the element is erased or not using the bitfield; if it is not erased, do nothing. If it is erased, read the jump value from the erased element's memory space and skip forward or backward (depending on the direction of iterator) the appropriate number of nodes, both in the element block and the bitfield.

This approach introduces 3 additional sets of operations per iteration via (1) branching operations when checking the bitfield, (2) bitmasking + bitshift to read the bits and (3) additional reads (from the erased element memory space). But it does reduce the memory overhead of the skipfield to 1 bit per-element. Unfortunately this also means the type has to be wide enough to fit both free list indexes and the jump-counting data - which means, assuming a doubly-linked free list, the type must be 32-bits or greater (assuming max block capacities of 255 elements, meaning the free list indexes can be 8-bit each), otherwise its storage will need to be artificially widened to 32 bits. There is a way around this though:

Bitfield + jump-counting pattern - complex variant:

- As with the simple variant, we have a bitfield indicating which elements are erased.
- If skipblock contains only one 1 element, there is no need to store jump-counting data - we only store free list data in the erased element memory space of the (singular) erased element.
- If skipblock is 2 elements long, we store the jump-counting data in the memory space of the 2nd erased element in the skipblock. In this way we avoid having to increase the alignment of very small types to accomodate storage of both the jump-counting data + free list data.
- If skipblock is > 2 elements, store the jump-counting data in the memory space of the 2nd erased element + the last erased element in the skipblock. The latter is used with -- iteration.

++ iteration becomes:

1. Increment both the current element pointer and bitfield index.
2. Check to see if we are now beyond the end of the element block. If so, go to step 3. If not, go to step 4.
3. Change element pointer to the start of the next element block, bitfield pointer to the next block's bitfield, and set bitfield index to 0.
4. Read bitfield[index]. If it is 0, ++ is finished.
5. If it is 1, read bitfield[index + 1] - if this is 0 (or beyond the end of the bitfield), increment both the element pointer and bitfield index, and go to step 7.
6. If bitfield[index + 1] is 1, dereference (element pointer + 1) to find the jump-counting value, and add this value to the element pointer and the bitfield index.
7. If we have already gone forward one block in step 3, ++ is finished. Otherwise check to see if we are now beyond the end of the element block. If so go to step 3. Otherwise ++ operation is finished.

-- iteration becomes:

1. Decrement both the current element pointer and bitfield index.
2. Check to see if we are now beyond the beginning of the element block. If so, go to step 3. If no, go to step 4.
3. Change element pointer to the back element of the previous element block, bitfield pointer to the previous block's bitfield, and set bitfield index to the capacity of that element block - 1.
4. Read bitfield[index]. If it is 0, -- is finished.
5. If it is 1, read bitfield[index - 1] - if this is 0 (or beyond the beginning of the bitfield), decrement both the element pointer and bitfield index and go to step 7.
6. If bitfield[index - 1] is 1, dereference the element pointer to find the jump-counting value, and subtract this value from the element pointer and the bitfield index.
7. If we have already gone back one block in step 3, -- is finished. Otherwise check to see if we are now beyond the beginning of the element block. If so go to step 3. If not -- operation is finished.

This approach involves a greater number of operations than the simple variant, but means that for a 16-bit type and max block capacities of 255 elements (meaning the free list indexes can be 8-bit), the type's storage will not need to be artificially widened in order to store both the doubly-linked free list and jump-counting data.

3. Erased-element location recording mechanism

There are two known valid approaches here; both involve per-element-block [free lists](#), utilizing the memory space of erased elements to form a list of erased locations. The first approach forms a free list of all erased elements. The second forms a free list of the first element in each skipblocks. The reference implementation currently uses the second approach, as discussed in [Design Decisions](#).

One cannot use a stack of pointers (or similar) to erased elements for this mechanism, as early versions of the reference implementation did, because this can create allocations during erasure, which violates the exception guarantees of `erase()` in the standard.

Why a global free-list won't work:

If a global (ie. not per-element-block) free list were used, pointers would be necessary instead of indexes, as finding the location of an erased element based on a (global) index would be $O(n)$ in the number of active blocks (counting each block's capacity as we went). This would increase the minimum bitdepth necessary for the hive element type to $\text{sizeof}(\text{pointer}) * 2$. A global free list would also decrease cache locality when traversing the free list by jumping between element blocks. Lastly, when a block was removed from the *active blocks* upon becoming empty, it would force an $O(n)$ traversal of the free-list to find all erased elements (or skipblocks) within that particular block in order to remove them from the free list. Hence a global free list is unacceptable for both performance and latency reasons.

Why the reference implementation uses a doubly-linked free list and where you can have a singly-linked one:

Previous versions of the reference implementation used a singly-linked free list of erased elements instead of a doubly-linked free list of skipblocks. This is possible with the high complexity jump-counting pattern, but not using the low complexity jump-counting pattern, as the latter cannot calculate the location of the start node of a skipblock from the value of a non-start node, but the high complexity variant can (see both of the jump-counting papers listed earlier for more details). But using free-lists of skipblocks is more efficient as it requires fewer free list nodes. In addition, re-using only the start or end nodes of a skipblock is faster because it never splits a skipblock into two skipblocks.

An example of why a doubly-linked free list is necessary for the low complexity jump-counting pattern, is erasing an element which happens to be between two skipblocks. In this case two skipblocks must be combined into one skipblock, and the previous secondary skipblock must be removed from that block's free list. If the free list is singly-linked, the hive must do a linear search through the free list, starting from the free list head, in order to find the skipblock prior to the secondary skipblock mentioned, to update that free list node's "next" index link. This is at worst $O(n)$ in the number of skipblocks within that block. However if a doubly-linked free list is used, that previous skipblock is linked to from the entry in the skipblock we have to remove, making the free list update constant-time.

Likewise when an erasure occurs just before the front of a skipblock (where the free-list data is stored), expanding the skipblock, the same scenario applies; for a singly-linked free list, one has to traverse the whole free list starting from the free list head, in order to find that skipblock's 'previous' free list node in order to update the previous node's 'next' link to point to the new start location of the changed skipblock. If the free list is doubly-linked we don't have to.

If the high-complexity jump-counting pattern is used, then we can calculate the start of a skipblock from the value of the erased skipfield node; and from the start nodes value we know the length of the skipblock. This means we can alter skipblocks using the information given to us by any node in the skipblock, not just the back/front nodes. Which in turn means we can make a free list from individual erased elements rather than skipblocks. And this in turn means there is no need to combine or update previously-existing free list entries in the examples above, and we can simply use a singly-linked free list instead of a doubly-linked one.

Keeping track of which blocks have erased elements:

So far I have largely been talking about how to keep track of erasures within element blocks, not about which blocks have erasures in them. In the reference implementation the latter is achieved by keeping an intrusive linked list of the groups whose blocks contained erasures, as mentioned previously. This increases group metadata memory usage by two pointers. Alternative methods include:

- a. Storing a vector of pointers to groups whose blocks contain erasures. To preserve `erase()` exception guarantees, this vector would have to be expanded upon insertion, not erasure, and thus its capacity should always be $\geq$ the number of active blocks. When an active block becomes completely empty of elements and is removed from the iterative sequence, its pointer in the vector would be swapped with the back pointer and popped. This approach reduces the additional memory cost down to 1 pointer per active block.
- b. Storing a record of the last one-or-more groups whose blocks have had erasures and then re-using from those groups during insertion, until they are full and then (on subsequent insertions) searching consecutive groups until a group with erasures is found. The records would be updated every time an erasure is made. The assumption is made that groups close to the recorded group(s) with erasures are more likely to have erasures, but this depends on the erasure pattern. This is the approach used by [plf::list](#), but it is efficient there because the main structure is a vector of metadata, hence the search phase is cache-friendly and fast even though the worst case scenario is $O(n)$ in the number of active blocks. In hive it would only be cache-friendly if one took the [vector-of-pointers-to-blocks approach](#) and stored the relevant metadata which indicates erasures, with the pointer, in the vector. The performance of this approach scales with the number of active blocks, so while it reduces the cost of keeping track of which active blocks have erasures down to 2 pointers per hive, it may reduce performance and increase latency variability during insertion due to the search phase, as described.
- c. If a [vector-of-pointers-to-blocks approach](#) is taken one could construct a low-complexity jump-counting skipfield which skips active blocks which *do not* contain erasures, and use this to find the first block with erasures in $O(1)$ time. This reduces the memory cost down to 1 `size_type` per active block.

Additional info for supporting small types

In the reference implementation we do not accommodate 1-byte types without artificially widening the storage the type uses to be `sizeof(skipfield_type) * 2` ie. 2 bytes. This is in order to accomodate the doubly-linked free list of skipblocks, which is expressed in pairs of prev/next indexes written to the erased element's memory location. Those indexes have to be able to address the whole of the specific element block, which means they have to be the same size as `skipfield_type`.

If an implementation wished to create an overload for 1-byte types such that there was no artificial widening of the element storage and the resultant wasted memory, there are 6 valid approaches I can see.

1. 15-element blocks + 4-bit skipfield

In this approach we reduce the skipfield bit-depth to 4-bits which in turn reduces the capacities of element blocks to 16 elements. We can do this via a [bit-field](#). We then continue business-as-usual and store our free list node data in the erased element memory space. Blocks are 15-elements wide rather than 16 elements as we need to reserve the value of `numeric_limits<skipfield_type>::max()` [as described above](#).

2. 15-element blocks + 1-bit skipfield

Here we take the [bitfield + jump-counting skipfield \(complex variant\)](#) approach. Again, since our block size is 15 elements, jump-counting data can be stored using 4-bit integers. So the free list's prev and next indexes can be stored within the memory space of a single 1-byte erased element, while the jump-counting data is stored in the memory space of both the second and last erased elements in the skipblock.

3. 255-elements-max blocks + 8-bit nextfield + 1-bit bitfield

Here, instead of artificially widening type storage so that we can store 2 bytes of free list data in it, we create a secondary 'next' field (1-byte array) to store the 'next' index of our free list, while the 'prev' index is stored in the erased element's memory space. We then follow the [bitfield + jump-counting skipfield \(complex variant\)](#) approach, storing 8-bit jump-counting data in the memory space of both the second and last erased elements in the skipblock.

4. 127-elements-max blocks + 7-bit nextfield combined with 1-bit bitfield

Same as the approach above, but instead of having a separate nextfield, we reduce it to 7 bits and combine it with the bitfield to make a singular 8-bit array. 7 bits reduces the max capacity of the element blocks.

5. 255-elements-max blocks + high-complexity jump-counting skipfield + 8-bit skipfield + singly-linked free list of erased elements (instead of skipblocks) (*Requires tech spec change*)

Here we substitute the low-complexity jump-counting skipfield for the high-complexity variant, which is at worst $O(n)$ in the number of skipfield nodes in a given block for insertion and erasure, so would require widening of their time complexity. However, our block size is at most 255 elements (with 255 corresponding skipfield nodes) which limits the number of operations. The high-complexity pattern allows us to have a singly-linked free list of erased elements instead of skipblocks for the [reasons described earlier](#). So we can store our 8-bit 'next' value in the erased-element memory space without having to widen the type bitdepth.

6. 15-element blocks + high-complexity jump-counting skipfield + 1-bit skipfield + singly-linked free list of erased elements (instead of skipblocks)

Here we take a [bitfield + jump-counting skipfield \(simple variant\)](#) approach and use it with a 4-bit high-complexity jump-counting skipfield. We can store both the jump-counting data + the free list 'next' value in the erased element memory space (since both are 4-bits). Since the block size is fixed at 15 elements, the high-complexity jump-counting approach is effectively $O(1)$.

However if we look at the memory cost of each of these approaches, only two are adequate. Assuming we take a 'vector-of-pointers' approach to implementation + a jump-counting skipfield of blocks-with-erasures (as described in the summary at the start of this appendix) + doing a few other things like allocating the element block and skipfield block in one allocation, and using the offset in capacity from the start of the element block to find the skipfield block (instead of storing a pointer to the skipfield block), we can reduce the per-block metadata down to around 40 bytes. Taking each of the approaches listed above as well as the reference implementation's approach, for 253 elements we would end up with the per-element memory cost being (not including unused skipfield/element block memory space):

1. 15-element blocks + low-complexity jump-counting skipfield + 4-bit skipfield + doubly-linked free list of skipblocks

Metadata cost: $17 * 40$ bytes

Skipfield cost: $253 * 4$ bits (.5 bytes)

Additional memory cost per element: 3.19 bytes.

2. 15-element blocks + low-complexity jump-counting skipfield + 1-bit skipfield + doubly-linked free list of skipblocks

Metadata cost: $17 * 40$ bytes

Skipfield cost: $253 * 1$ bits (.125 bytes)

Additional memory cost per element: 2.81 bytes.

3. Maximum 255-element blocks low-complexity jump-counting skipfield + 8-bit nextfield + 1-bit skipfield + doubly-linked free list of skipblocks

Metadata cost: $1 * 40$ bytes

Nextfield cost: $253 * 1$ byte

Skipfield cost: $253 * 1$ bits (.125 bytes)

Additional memory cost per element: 1.28 bytes.

4. Maximum 127-elements blocks + 7-bit low-complexity jump-counting skipfield + nextfield combined with 1-bit bitfield + doubly-linked free list of skipblocks

Metadata cost: $2 * 40$ bytes

Combined skipfield/nextfield cost: $253 * 1$ bytes

Additional memory cost per element: 1.32 bytes.

5. Maximum 255-element blocks + high-complexity jump-counting skipfield + 8-bit skipfield + singly-linked free list of erased elements

Metadata cost: $1 * 40$ bytes

Skipfield cost: $253 * 1$ bytes

Additional memory cost per element: 1.16 bytes.

6. 15-element blocks + high-complexity jump-counting skipfield + 1-bit skipfield + singly-linked free list of erased elements

Metadata cost: $17 * 40$ bytes

Skipfield cost: $253 * 1$ bit (.125 bytes)

Additional memory cost per element: 2.97 bytes.

7. (reference implementation, ie. artificially widen the type storage to 16-bits) Maximum 255-element blocks + low-complexity jump-counting skipfield + 8-bit skipfield + doubly-linked free list of skipblocks

Metadata cost: $1 * 40$ bytes

Widening cost: $253 * 1$ bytes

Skipfield cost: $253 * 1$ bytes

Additional memory cost per element: 2.16 bytes.

Clearly the third, fourth and fifth approaches are the only viable ones, if memory waste is a concern. They also allow allocating larger numbers of elements contiguously and reduce the number of allocations/deallocations, which improves insert/erase/iteration performance compared to other non-reference approaches. Since only the third and fourth don't require changing the API, they would be the better approaches, between those two the third has better cache locality due to larger element blocks, and also less memory waste so that's the preferred one.

An implementation guide using the vector-of-pointers-to-blocks approach

I will give the summary first, then show in detail, how we get there and why some approaches aren't possible. When I talk about vectors here I'm not really talking about `std::vector`, more likely a custom implementation.

Summary

Like in the reference implementation, there are structs (referred to herein as 'groups') containing both an element array + skipfield array + array metadata such as size, capacity etc. Each group has it's own erased-element free list just like the reference implementation.

The hive contains a vector of pointers to groups (referred to herein as a 'group-vector'). The group-vector contains 2 extra pointers, one at the front of the active group pointers and one at the back of the active group pointers, each of which has it's own location in memory as it's value (these are referred to herein as the *front* and *back* pointers).

Each allocated group also contains a reverse-lookup pointer in it's metadata which points back to the pointer pointing at it in the group-vector. While this is used in other operations it is also used by iterator comparison operators to determine whether the group that iterator1 is pointing to is later than the group that iterator2 is pointer to, in the iterative sequence.

An iterator maintains a pointer to the group, the current element and the current skipfield location (or just an index into both the element and skipfield arrays). When it needs to transition from the end or beginning of the element array to the next or previous group, it takes the value of the reverse-lookup pointer in the current group's metadata and increments or decrements it respectively, then dereferences the new value to find the next/previous group's memory location.

If the value of the memory location pointed to is `nullptr`, it increments/decrements again till it finds a non-`nullptr` pointer - this is the next block. If the value of the memory location pointed to is equal to the memory location, the iterator knows it has reached the front or back pointer in the vector, depending on whether it was decrementing or incrementing respectively. This is the only purpose of the front and back pointers, to inform the iterator of boundaries (see later for more details).

When a group becomes empty of non-erased elements, it is either deallocated or retained for future insertions by copying it's pointer in the group-vector to an area past the back pointer, depending on implementation. Either way it's original pointer location in the group-vector is `nullptr`'ed.

There is a hive member counter which counts the number of `nullptr`'ed pointers. If it reaches a implementation-specific threshold, a `erase_if` operation is called on the vector, removing all `nullptr` pointers and consolidating the rest. Subsequently (or as part of the `erase_if` operation) the groups whose pointers have been relocated have their reverse-lookup pointer updated. The threshold prevents (a) iterator `++/--` functions straying too far from being $O(1)$ amortized in terms of number of operations and (b) too many `erase_if` operations occurring.

Likewise for any splice operation, when source groups become part of the destination, the destination group-vector gets pointers to those groups added, and the reverse-lookup pointers in those groups get updated. All reverse-lookup pointers get updated when the vector expands and the pointers are reallocated.

To keep track of groups which currently have erased-element locations ready to be re-used by insert, we can either keep the reference implementation's intrusive-list-of-groups-with-erasures approach, or we can remove that metadata from the group and instead have a secondary vector of `size_type` with the same capacity as the group-vector, containing a jump-counting skipfield.

In that skipfield we maintain a record of runs of groups which do Not currently have erased element locations available for reuse, so that if there are any such groups available, a single iteration into this skipfield will take us to the index corresponding to that group in the group-vector. And if there are no such groups available, that same iteration will take us to the end of the skipfield. This approach replaces 2 pointers of metadata per-group with one `size_type`.

If insertion determines that there are no groups with erasures available, it can (depending on implementation) either check a hive member counter which counts the number of nullptr'd pointers, and if it's not zero, linear-scan the group-vector to find any nullptr locations and reuse those to point to a new group - or it could just move the back pointer forward by 1 and reuse that location to point to a new group (relying on the occasional erase_if operations to clean up the nullptr pointer locations instead, and running erase_if itself only if the vector has reached capacity). If the implementation has retained a pointer to an empty group past the back pointer (a group made empty by erasures) it could reuse that at this point.

Alternative strategies within the above approach:

1. One can, instead of storing all block metadata with the block, store some of it with the pointer in the vector-of-pointers, or even in it's own vector (following a struct-of-arrays-style formation).
2. For example, one could store capacity and size with the pointer in the vector instead of in the group, and set capacity to 0 when a group is removed. The iterator could then use the value of capacity, instead of nulling the pointer, to indicate a vector entry they need to skip when iterating. The hive could then use the pointers themselves to form a free list of empty vector positions, and re-use these when a new group needs to be created.
3. We could store jump-counting data with the pointer to skip over runs of erased groups in O(1) time, and update this on the fly.
4. We could store the per-block free list of erased elements head in a separate vector, and linearly-scan for the first free list head without a 'no erasures' value (if copying the current implementation, this value is `numeric_limits<skipfield_type>::max()`). Again we run the risk of latency due to branching, but this consumes no additional memory in order to record which blocks have erasures. This would likely require specification complexity adjustment.

How we get there, starting from the simplest approach

The simplest idea I had for the alternative (non-reference-implementation) approach was, a vector of pointers to allocated element blocks. In terms of how iteration works, the iterator holds a pointer to the vector-of-pointers (the structure, not the vector's internal array) and an index into the vector-of-pointer's array, as well as a pointer/index to the current element. The hive itself would also store a pointer to the vector structure, allocating it dynamically, which makes swapping/moving non-problematic in terms of keeping valid iterators (if the hive stores the vector as a member, the iterator pointers to the vectors get invalidated on swap/move).

When the end of a block is reached by the iterator, if it hasn't hit end() you add 1 to the vector-of-pointers index in the iterator and continue on to the next block. Since the iterator uses indexes, reallocation of the vector array upon expansion of the vector-of-pointers doesn't become a problem. However it is a problem when a block prior to the current block that the iterator is pointing at, becomes empty of elements and has to be removed from the iterative sequence. If the pointer-to-that-block gets erased from the vector-of-pointers, that will cause subsequent pointers to be relocated backward by one, which in turn will make iterators pointing to elements after that block invalid (because the relocation invalidates the stored block indexes in the iterators).

Substituting a swap-and-pop between the erasable pointer and the back pointer of the vector-of-pointers, instead of erasing/relocating, doesn't solve this problem, as this produces unintuitive iteration results when an iterator lies between the back block and the block being erased (suddenly there is a bunch of elements behind it instead of in front, so forward iteration will miss those), and it also invalidates iterators pointing to elements in the (previously) back block.

So at this point we only have two valid approaches, A & B.

Approach A:

Here we have to think it terms of what's efficient, not what necessarily lowers time complexity requirements. Basically instead of erasing pointers to the erased blocks from the vector, we mark them as `nullptr` and the iterator, when it passes to the next block, skips over the `nullptr` pointers. This is the opposite of what we try to do with the current approach in the reference implementation (remove blocks from the iterative linked-list-of-blocks sequence) because with the current approach, those blocks represent a latency issue via following pointers to destinations which may not be within the cache. With a vector approach however, it makes no difference to latency because the next pointer in the vector chunk already exists in the cache in, at a guess, 99% of cases. You could, potentially, get a bad result when using a CPU with poor branch-prediction-recovery performance like core2's (because this approach introduces a branching loop), when you have a close-to-50/50 random distribution of `nullptr`'s and actual pointers to blocks. But since blocks are generally going to be many factors fewer than elements within those blocks, this is not likely to be a major performance hit like a boolean skipfield over elements would be, even in that case.

In terms of re-using those `nullptr` pointers, we can't do a free-list of pointers because then during iteration we would have no idea which pointer was a pointer to a block and which a pointer to another free-list item - so instead we simply have a `size_type` counter in the hive metadata which counts the number of erased pointers currently in the vector-of-pointers. When we reach the capacity of existing element blocks and need to create a new block upon insert, we check the counter - if it's not zero (ie. time to create new a block pointer at the back of the vector), scan manually along the vector-of-pointers until you hit a `nullptr` and re-use that (same logic as above as to why this isn't a latency/performance issue) and decrement the 'erased pointer' counter.

Since insertion location is unspecified for hive, inserting a block randomly into the middle of the iterative sequence causes no fundamental issues, is the same as re-using an erased element location during insertion.

Approach B:

If one is concerned about strict time complexity, and less concerned about real world effects of that time complexity, you can basically have a jump-counting skipfield for the vector-of-pointers (secondary vector of `size_type` with a low-complexity jump-counting skipfield).

This means (a) iterators can skip over pointers to erased blocks in $O(1)$ time and (b) the memory locations of the pointers to erased blocks can be used to form a free list of reusable pointers. So this eliminates both of the non- $O(1)$ aspects of Approach A, though whether or not this is faster in-practice comes down to actual benchmarking.

Metadata and probable advantages of either approach:

I've left out block metadata (size, capacity, erased element free list head, etc) in the descriptions above to simplify explanation, but for approach A we would probably want block metadata as part of a struct which the vector-of-pointers is pointing to (struct contains the element block too), so that the non- $O(1)$ linear scans over the vector-of-pointers are as fast as possible.

For approach B we would probably want the vector-of-pointers to actually be a vector-of-struct-metadata, with the pointer to the element block being one of the pieces of metadata. We could also do a 'struct of arrays' approach instead, depending on the performance result.

Both approaches eliminate the need for the 'block number' piece of metadata since we get that from the vector-of-pointers for free. They also eliminate the need for prev-block/next-block pointers, though this is offset by the need for the vector of pointers in approach A and for approach B the secondary skipfield - but still a reduction of 2 pointers to 1 pointer/`size_type`.

The intrusive-list-of-blocks-with-element-erasures from the reference implementation could in this approach be replaced with a [low-complexity jump-counting skipfield](#) (an additional vector of `size_type`) which, instead of denoting runs of erased blocks, denotes runs of blocks with no prior element erasures (this would include erased

blocks, since these are not usable for insert). If there were no prior erasures, iterating over this skipfield would jump directly to the end of the skipfield on the first iteration. This further reduces the memory use per-block of recording down from 2 pointers in the reference implementation, to 1 `size_type`.

Alternatively if we go the vector-of-metadata-structs route, and don't mind doing a non- $O(1)$ linear scan upon insert, we can linear-scan the erased-element-free-list-head metadata of each block to find blocks with erasures, subsequently eliminating additional memory use for recording blocks-with-erasures entirely. This approach would be benefited by splitting the vector-of-metadata-structs into a struct-of-vectors for each metadata item.

But `splice()` changes our requirements:

`Splice` requires that iterators to the source and destination hive's elements not be invalidated when the source hive's elements are transferred to the destination hive.

If we take a vector-of-pointers/vector-of-metadata approach, and our iterators use indexes into that vector, those indexes will be invalidated during `splice`, as the source vector's contents must be transferred into the destination vector. Further, the pointer-to-vector which the iterator must hold in order to transition between blocks, would also be invalidated during `splice` for iterators pointing to source elements - which means that swapping from vectors to deques and using pointers instead of indexes within the iterators, would not help.

The solution is unintuitive but functional: the iterator becomes much the same as the reference implementation: either 3 pointers, one to the block+metadata struct, one to the element and one to the skipfield, or 1 pointer (to the struct) and one index (into the element and skipfield blocks respectively). We add a "reverse-lookup" pointer into the element block's metadata which points back at the pointer pointing to it in the vector (we could have a pointer to the vector block + an index, but this consumes more memory so we'll just say a pointer to the vector position for the remainder of this text). When the iterator needs to transition blocks it follows the pointer out to the vector and increments or decrements as necessary. When `splice()` is run it alters the reverse-lookup pointers in each of the source's blocks such that they point to the correct pointers in the destination's vector. If the vector has to expand either during `splice` to accomodate the source blocks, the reverse-lookup pointers will need to be updated for all blocks.

Neither move nor swap are required to update these pointers, since those operations will simply swap the members of the vector including the pointer to the dynamically-allocated internal array, and neither the block metadata nor the iterators contain pointers to the vector itself. As such we no longer need to dynamically-allocate the vector-of-pointers in the hive and can just have it as a member.

The solution does not entirely rule out Approach B (vector of metadata structs) in the above sections, but simply means that the reverse-lookup pointer *must* be stored with the element block, while other metadata may be stored either with the element block, or in the vector, or in separate vectors (ie. in a struct-of-arrays style), as desired or is determined to perform well.

Also: this solution allows us to fully erase entries from the vector and relocate subsequent entries, since we're no longer relying on indexes within the iterators themselves to keep track of the block pointer location in the vector. An implementation can choose whether or not they want to consolidate the vector after a block erasure, and might want to have a maximum threshold of the number of erased entries in the vector before doing so (or possibly the number of Consecutive erased entries). This prevents the number of operations per `++/--` iterator operation from becoming too high in terms of skipping erased entries and causing latency. But more importantly, it keeps `++/--` iterator operation at $O(1)$ amortized (and removes any performance problems relating to poor branch-prediction-recovery as described earlier).

The vector erase operation (`erase_if` if we're following a threshold approach and consolidating multiple erased entries at the same time) would process block metadata and adjust the reverse-lookup pointers for each block to the new values. Likewise when insertion triggers an expansion of the vector, the reverse-lookup's get updated (if a deque is used instead of a vector, the latter is unnecessary as no reallocation takes place upon insert).

Lastly, we need a way for the iterator to figure out if it's at the beginning or end of the vector-of-pointers respectively. While we could store a pointer to the vector itself in the block metadata as well, this is being wasteful memory-wise. A less wasteful solution is to have 1 pointers at the beginning and end of the vector-of-pointers which have special values ie. two pointers per hive instead of one pointer per-block. The special value (since `nullptr`ing the element block pointer is already taken) can be the address of the pointer location itself, since this is a unique address. If we take the alternative approach described in the summary of using the capacity metadata to indicate erased blocks instead of the pointer, we can instead use `nullptr` for the front and back pointers.

iterator greater/lesser/spaceship operators (>/</>=/etcetera):

Now we lack a `group_number` metadata entry but we also lack a way to obtain the group number from the vector-of-pointers, since neither the block metadata nor the iterator currently store a pointer to the vector (and the iterator can't, since the pointer might get invalidated and the iterator can't get automatically updated by the container).

Luckily however, we don't need to know the group number for these operations to work; we only need to know if one group is later in the sequence than the other, and since we're storing a reverse-lookup pointer to the pointer in the vector-of-pointers, when comparing to see if `it1 < it2` we only need to check whether `it1->block_metadata->reverse-lookup < it2->block_metadata->reverse-lookup`.

If we use a deque-of-pointers-to-blocks instead of a vector we can't do the above, as the pointers are not guaranteed to point to locations within the same block. So for a deque we need have to store pointers to the deque in each block in order to get block numbers, which as mentioned is wasteful. We could however remove the back and front pointers in the deque-of-pointers at that point, as the iterator could use the pointer to the vector to find the front and back of the deque, instead.

That's all. This is probably not the only approach possible when not using the reference implementation, but it certainly will work.